

clusterAI

Ciencia de datos en ingeniería industrial

UTN BA

curso I5571

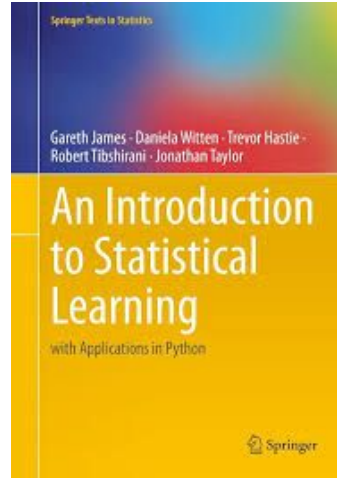
clase_03: Clasificación



agenda clase03: aprendizaje supervisado

- Aprendizaje Supervisado
- Clasificación, binary class, multiclass
- Performance metrics (Sens, Spec, ROC)
- Learning curve
- Clasificadores: KNN
- Train, validation, test
- Cross validation

Lectura sugerida



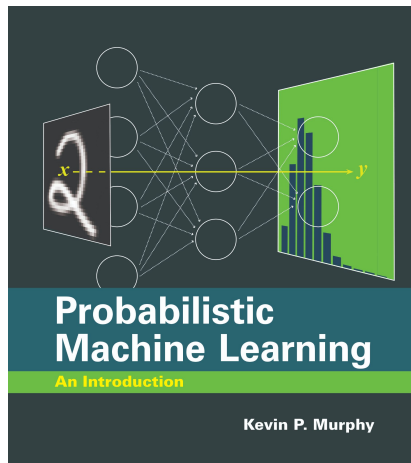
Introduction to Statistical Learning (with applications in Python)

<https://www.statlearning.com/>

https://hastie.su.domains/ISLP/ISLP_website.pdf.download.html

- Capitulo 1 -> completo
- Capitulo 2 -> página 15 a 34 (Introducción a Clasificación)
- Capítulo 4 -> página 135 a 144 (Regresión Logística)

Lectura sugerida



Probabilistic Machine Learning: An Introduction

<https://probml.github.io/pml-book/book1.html>

- Capítulo 1 Introducción
 - 1.1: What is machine learning?
 - 1.2: Supervised Learning
- Capítulo 11 -> Logistic Regression

Machine Learning (ML)

```
graph TD; ML[Machine Learning (ML)] --> Supervised[Supervised]; ML --> Unsupervised[Unsupervised];
```

Supervised

Unsupervised

Particularmente en este curso vamos a poner foco en el aprendizaje **supervisado** y el aprendizaje **no-supervisado**. Estos dos enfoques suelen ser los más populares y prácticos para la mayoría de los problemas.

Aprendizaje Supervisado

Se compone de un conjunto de entrenamiento $\mathcal{A}$ y un conjunto de testeo disjunto $\mathcal{T}$ de pares de entrada-objetivo $(x_0, y_0), (x_1, y_1), \dots, (x_N, y_N)$, donde $x_i \in \mathcal{X}$ e $y_i \in \mathcal{Y}$ se denominan espacio de entrada y espacio de destino, con $1 \leq i \leq N$, dado N el numero de elementos en los conjuntos.

Se supone que los pares de muestra de $\mathcal{A}$ y $\mathcal{T}$ se generan independientemente a partir de la misma distribucion de entrada-objetivo $\mathcal{D}_{\mathcal{X} \times \mathcal{Y}}$. En ocasiones, se asigna un conjunto de validacion adicional del conjunto de entrenamiento para validar el rendimiento del algoritmo de aprendizaje durante el entrenamiento y se utiliza tipicamente para decidir cuando detener el entrenamiento y evitar el sobreajuste.

Las tareas de aprendizaje supervisado se dividen en clasificacion y regresion cuando el espacio objetivo $\mathcal{Y}$ es un conjunto discreto de K clases y cuando el espacio objetivo $\mathcal{Y}$ es un subconjunto de numeros reales, respectivamente. Es decir, en tareas de clasificacion y regresion, los modelos ML aprenden una funcion de mapeo $f : \mathbb{R}^d \rightarrow \{1, \dots, K\}$, $K \subset \mathbb{N}$ y $f : \mathbb{R}^d \rightarrow \mathbb{R}$, respectivamente, donde $\dim(\mathcal{A}, \mathcal{T}) = d$.

Aprendizaje Supervisado

El objetivo del aprendizaje supervisado es minimizar una medida de pérdida específica de la tarea $\mathcal{L}$, denominada *loss function*. Idealmente, se desearía minimizar $\mathcal{L}$ en todo el $\mathcal{D}_{\mathcal{X} \times \mathcal{Y}}$. Sin embargo, esto requeriría conocer la forma analítica de la distribución o, en la práctica, tener acceso a un número infinito de pares de muestras. En su lugar, se utiliza el conjunto finito $\mathcal{A}$ para minimizar la llamada *training loss* $\mathcal{L}_{\mathcal{A}}$:

$$\mathcal{L}_{\mathcal{A}} = \frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} \mathcal{L}(f(x_i, \theta_t), y_i) \quad (1)$$

donde f y θ se refieren a la función de mapeo $f : X \rightarrow Y$ que el modelo ML debe aprender, y a sus parámetros, respectivamente.

Una vez finalizado el entrenamiento, el conjunto $\mathcal{A}$ se reemplaza por $\mathcal{T}$ en 1, lo que produce una medida $\mathcal{L}$ denominada *test loss*. La minimización de $\mathcal{L}_{\mathcal{T}}$ es, en la práctica, el verdadero objetivo del aprendizaje supervisado, ya que se busca un buen rendimiento en pares de muestras nunca antes vistos.

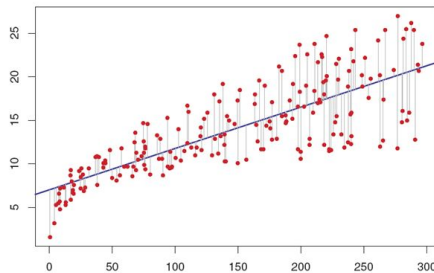
Aprendizaje Supervisado

La capacidad de transferir el conocimiento aprendido del conjunto $\mathcal{A}$ al conjunto $\mathcal{T}$ se denomina *generalización* y se basa en el supuesto de que $\mathcal{D}_{\mathcal{X} \times \mathcal{Y}}$ existe y $\mathcal{A} \cap \mathcal{T} = \emptyset$.

En particular, para tareas de clasificación, los modelos ML pueden diferenciarse en *discriminantes* o *probabilísticos*. Los enfoques discriminantes, como SVM, devuelven directamente la clase objetivo estimada para cada x_i después de que el modelo ML defina los límites entre las clases. Por el contrario, los enfoques probabilísticos primero determinan para cada muestra x_i las probabilidades condicionales $Pr(y_i = K | x_i, \theta)$ de las K clases, y luego se selecciona la clase más probable como la estimación $\hat{y}_i$:

$$\hat{y}_i = z(x_i, \theta) = \arg \max_K Pr(y_i = K | x_i, \theta) \quad (2)$$

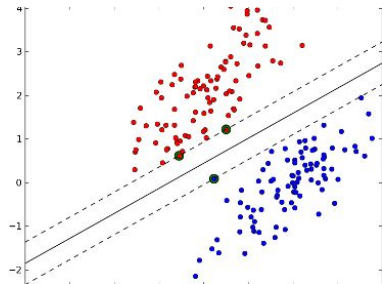
Regression



Y is a real number

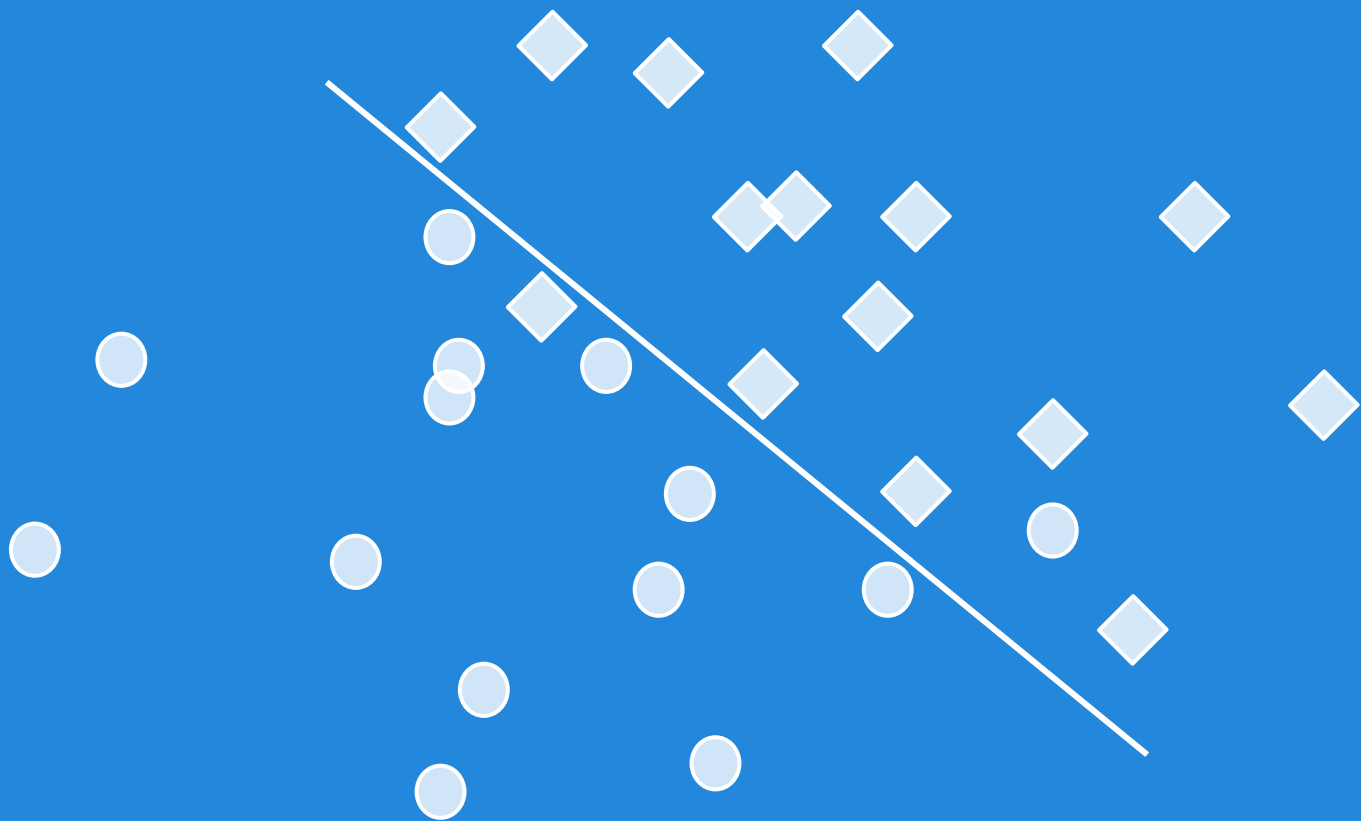
$$y \subseteq \mathbb{R}$$

Classification



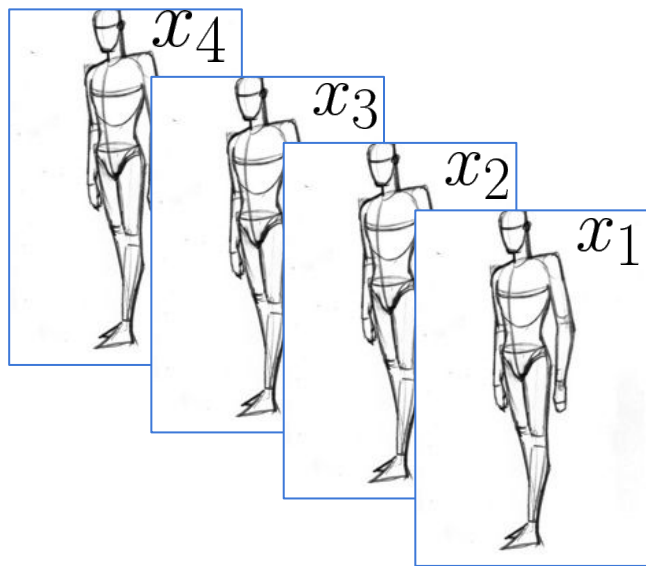
Y is categorical

$$y \in \{-1, 1\}$$



Aprendizaje Supervisado: Clasificación

Samples and Features



$$X_i = [x_{i1}, x_{i2}, x_{i3}, \dots, x_{id}]$$



$$\mathcal{X} \in \mathbb{R}^d$$

Para que los modelos de machine learning puedan aprender es necesario exponerlos a un conjunto de instancias/samples muestreados de una distribución de probabilidad compleja desconocida. Cada instancia/sample/muestra está caracterizada por un conjunto de features/variables/dimensiones.

Cada sample puede verse como un vector de dimensión d.

Aprendizaje supervisado: dataset

$$S = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

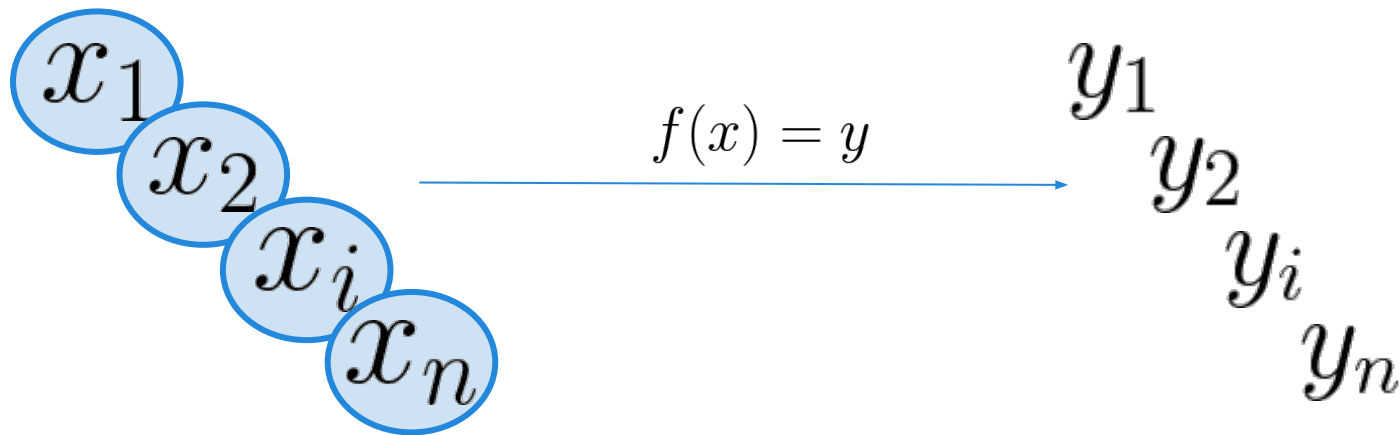
$$x \in \mathbb{R}^d \quad y \in \{-1, 1\} \quad f(x) = y$$

El enfoque de aprendizaje supervisado se basa en disponer datos ordenados en un dataset **S** en pares de instancias y etiquetas (samples 'x' & labels 'y'). Las instancias/muestras son vectores d-dimensionales de variables aleatorias i.i.d. (independientes e idénticamente distribuidos). Las etiquetas se suponen variables dependientes que pueden tomar valores discretos (clases) o continuos a partir de distintos valores de x mediante una función f(x) llamada 'ground truth' o función objetivo tal que f(x) = y. Es decir que f(.) explica la relación entre 'x' (input) e 'y' (output) Como la realidad es compleja generalmente no conocemos la verdadera f(x), por lo que trataremos de aproximarla o **aprenderla**.

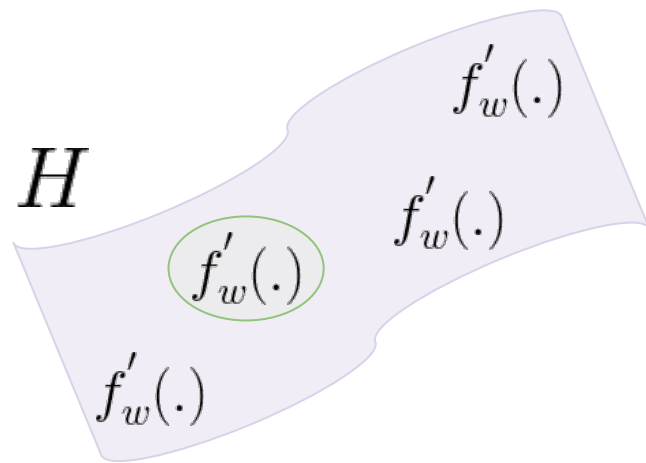
Aprendizaje supervisado: fundamentos

$$S = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

$$\mathcal{X} \in \mathbb{R}^d \quad y \in \{-1, 1\} \quad f(x) = y$$



Hipotesis



$$H = \left\{ f_w^1(\cdot), f_w^2(\cdot), \dots, \textcircled{f'_w(\cdot)}, \dots \right\}$$

Para aproximarnos a la verdadera $f(x)$ vamos a buscar alguna función $f(\cdot)$ dentro de un espacio de hipótesis que contiene muchas funciones $f(\cdot)$. De todas las funciones disponibles dentro del espacio de hipótesis H vamos a tratar de encontrar alguna que explique lo mejor posible la relación entre el input 'x' y el output 'y'. La función que vayamos a buscar estará caracterizada por **parámetros (w)** que pueden tomar distintos valores. Entonces existirá una combinación de parámetros que determinen una $f'(\cdot)$ que se aproxime a la verdadera $f(\cdot)$ mas que otras $f'(\cdot)$.

Aprendizaje supervisado

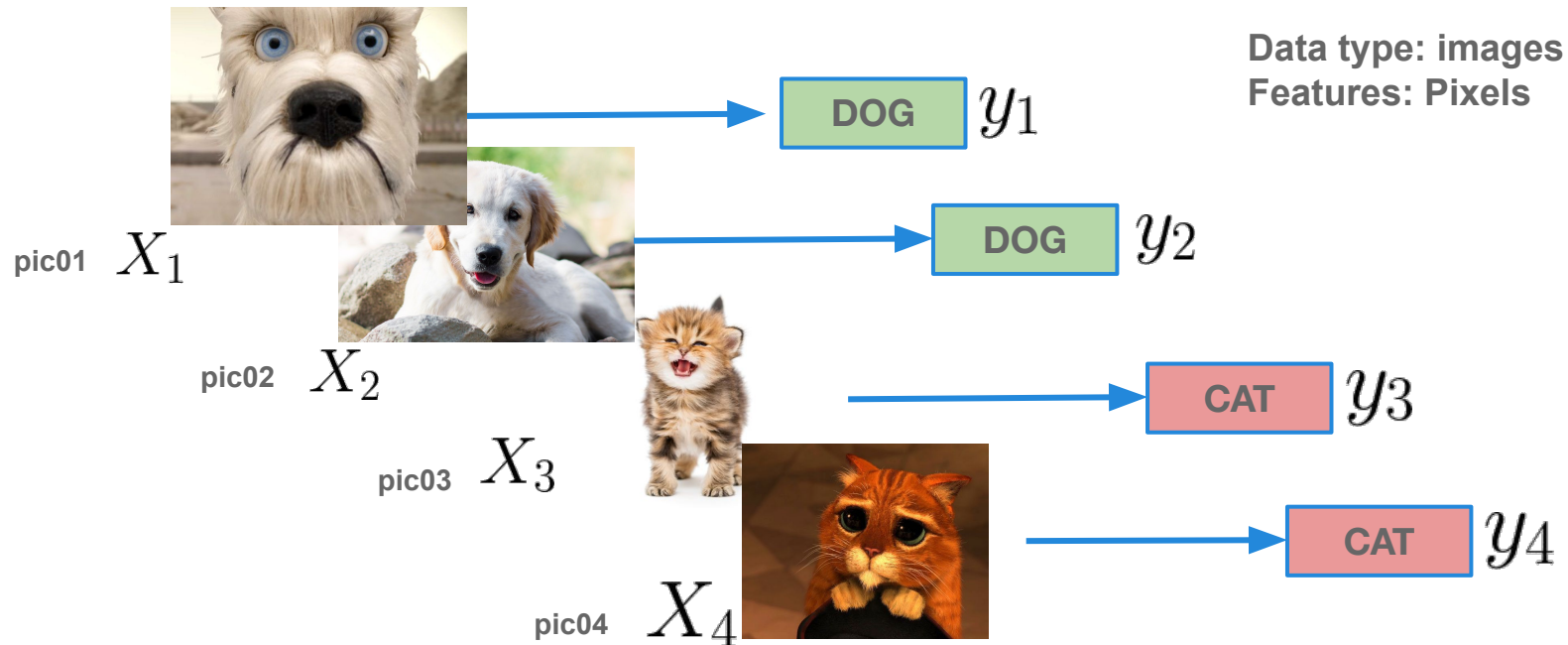
$$f(x) = y \quad \hat{f}(x) = \hat{y} \quad L(y, \hat{y})$$

Desconocida

$$\min_w L(y, \hat{y}) = \min_w L(y, \hat{f}(x))$$

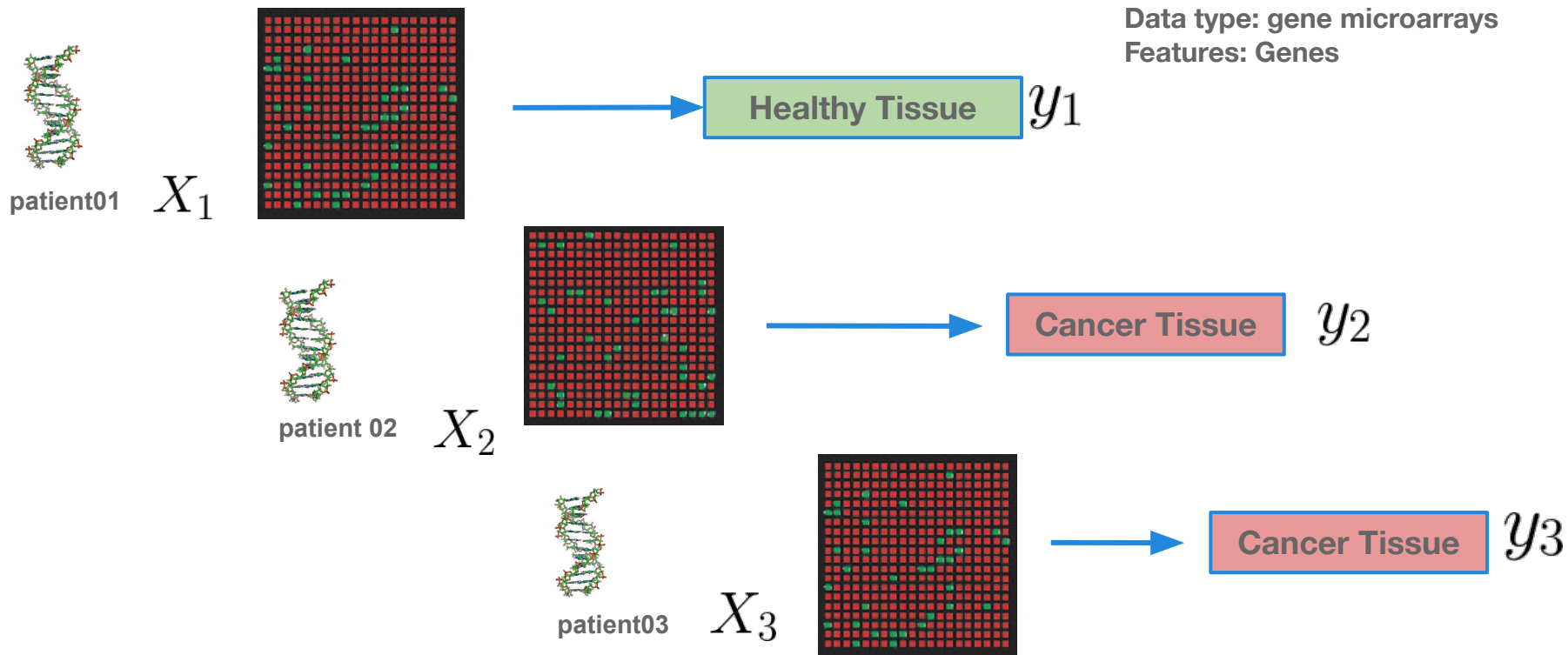
Suponiendo que tanto el dataset de sample-features y las etiquetas están disponibles $s=(x,y)$ vamos a **aprender los parámetros w que definen** una función $f'(x)$ que explique lo mejor posible la relación $(x \rightarrow y)$. Es decir que aprenderemos una función que tomando como input las variables aleatorias “ x ” genere un output y' lo más similar a las etiquetas “ y ” dadas. Para poder medir cuán similares son las etiquetas generadas por la función aprendida $f'(x)$ utilizaremos una función $L(y,y')$ de Costo o Pérdida (Loss function) que tomará valores altos cuando y sea muy distinto de y' . Por el contrario cuando y sea muy parecido a y' la función de costo tomará valores bajos. Por esta razón buscamos **minimizar** la función de costo. En otras palabras, el aprendizaje supervisado puede plantearse como un problema de optimización llamado **Empirical Risk Minimization**.

Aprendizaje supervisado: clasificación

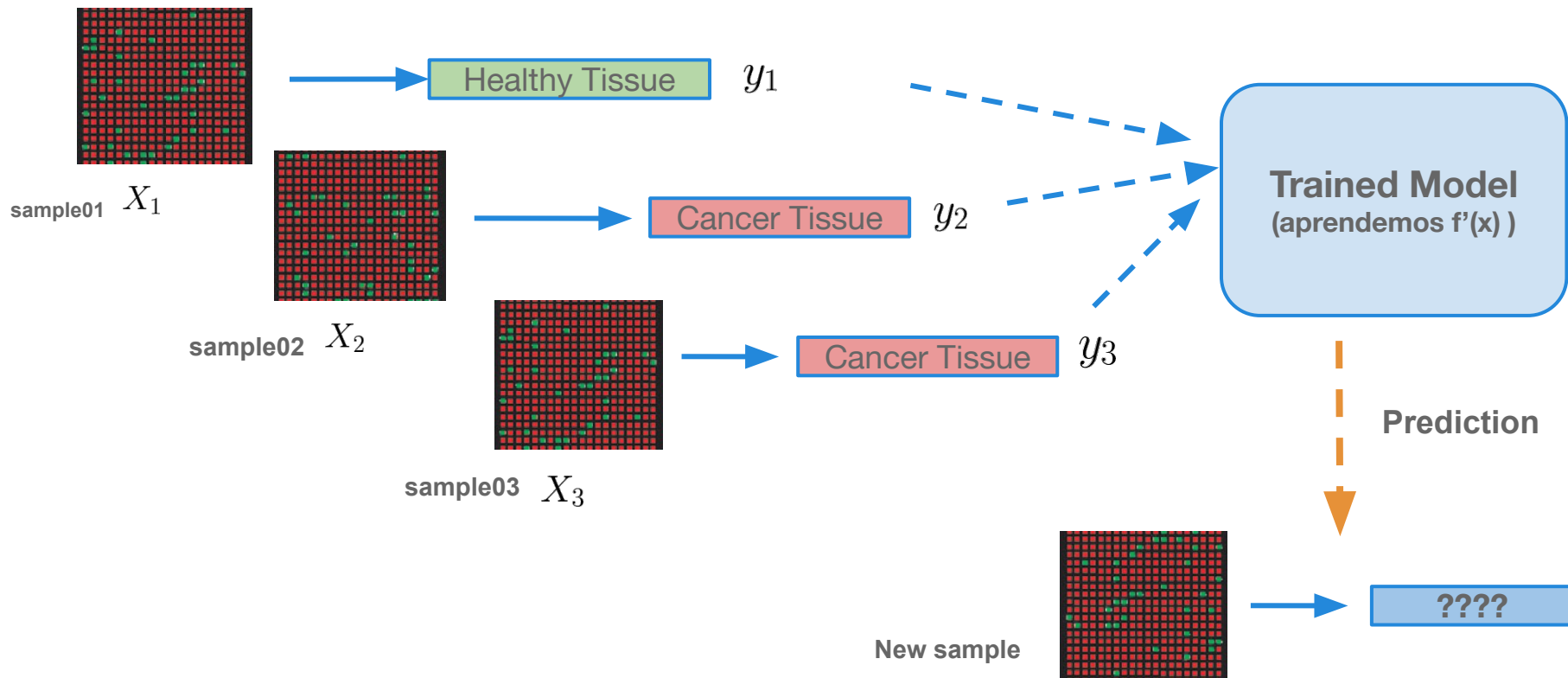


Cada muestra está asociada a una etiqueta categórica (clase) confirmada por un humano.

Aprendizaje supervisado: clasificación



Aprendizaje supervisado: clasificación



Clasificación: Frontera de decision lineal

Hiper-plano separador

$$f(x) = w^T x + b = 0$$

Funcion de decision

$$D(x) = \text{sign}[w^T x + b]$$

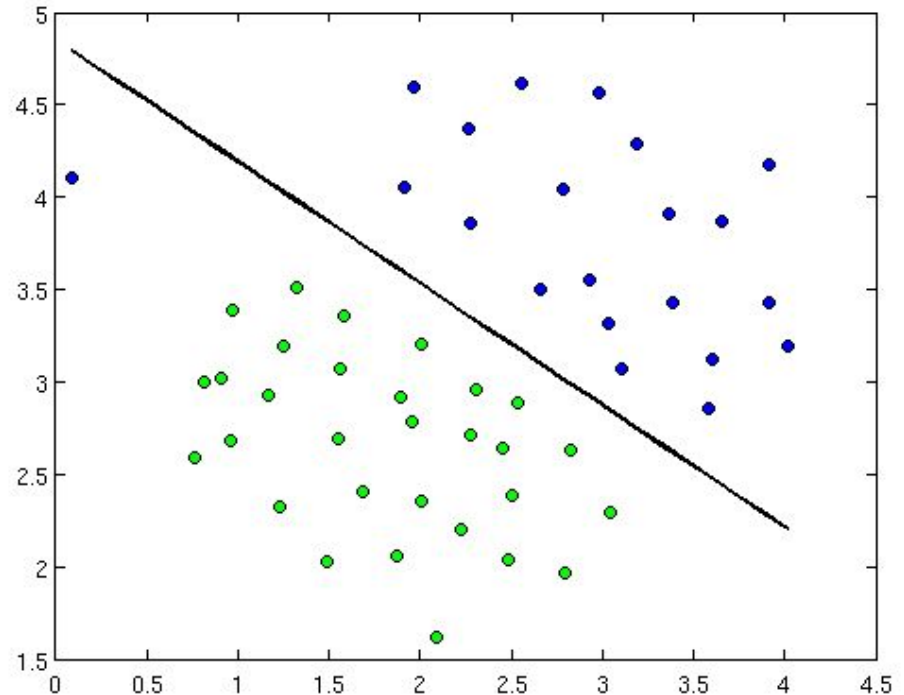
Existen muchos tipos de funciones de decisión. La familia de funciones más conocida es la de las funciones lineales. Estas funciones son hiper-planos caracterizados por parámetros $\mathbf{w}$ (vector $w=[w_1 \dots w_d]$) que determinarán cómo se posiciona la frontera de decisión en el hiper-espacio de dimensión d . En la clasificación binaria la función de decisión asignará un valor de $\mathbf{y=1}$ o $\mathbf{y=-1}$ según de que lado del hiper-plano se posicionen las muestras $\mathbf{x}$.

Funciones de decisión

Una función de decisión toma un vector input X (sample) con “d” features, y le asigna una de las K clases, llamada C_k .

- Cuando $C_k = 2 \rightarrow$ binaria
- Cuando $C_k > 2 =$ multiclase

En el ejemplo de la derecha tenemos dos dimensiones y dos clases. La función de decisión es una función lineal que clasifica casi todas las muestras bien menos una sola (dependiendo del lado que se encuentre la muestra respecto al clasificador).

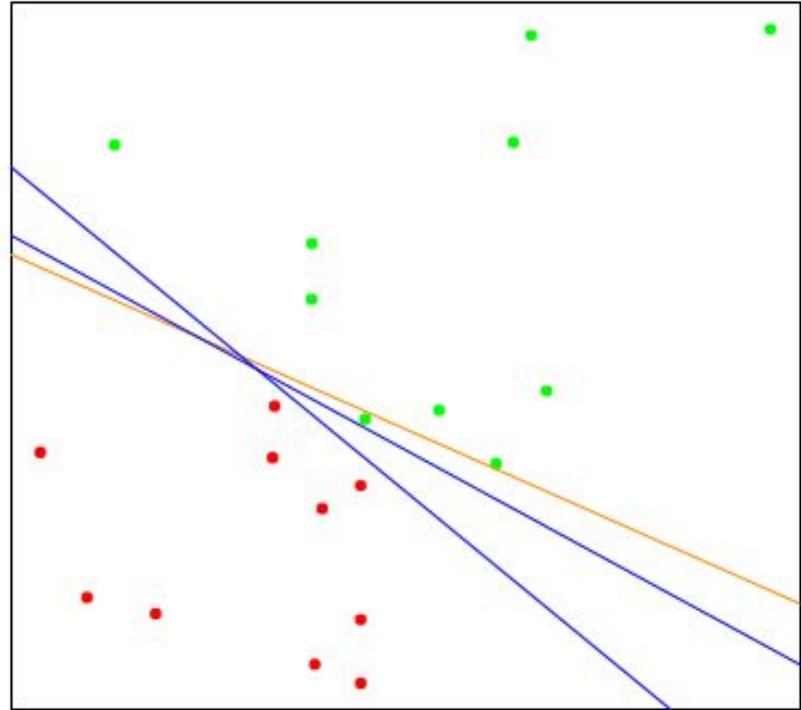


Funciones de Decisión

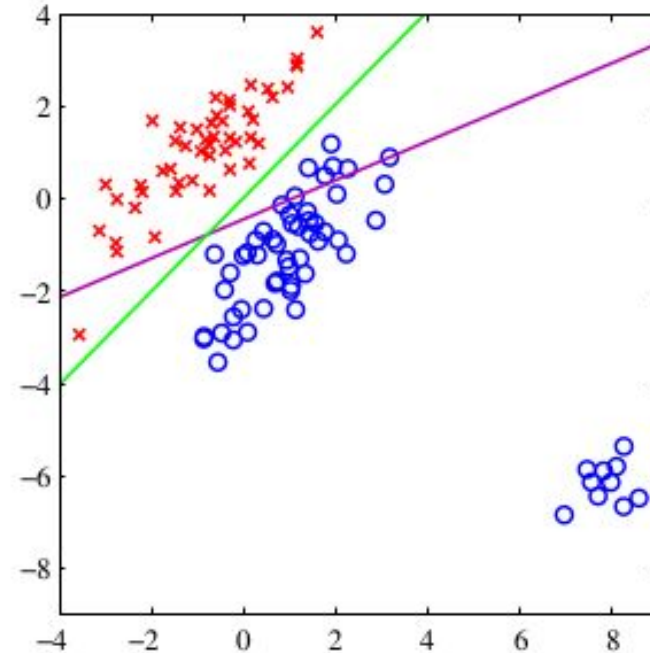
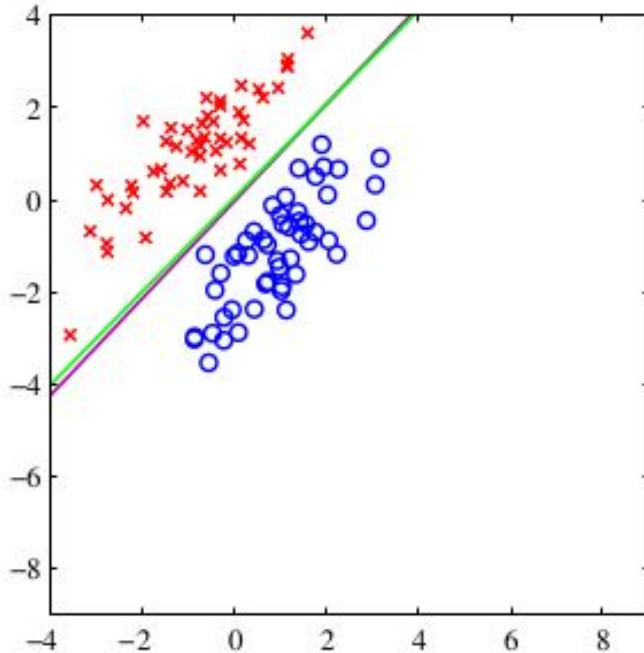
Para un mismo set de datos etiquetados, distintos modelos pueden generar **distintas** funciones de decisión (decision rules).

Algunos modelos generarán funciones de decisión más **sencillas** y otros aprenderán funciones más **complejas**.

La complejidad de la función a aprender dependerá de la complejidad de la **distribución** de las muestras de entrenamiento.



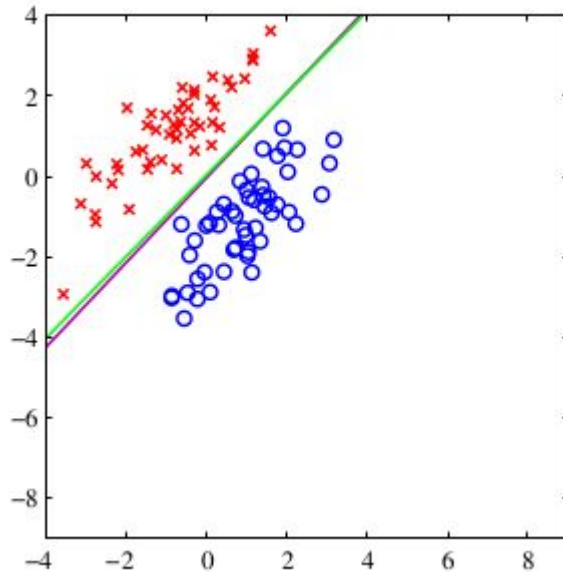
Funciones de Decisión



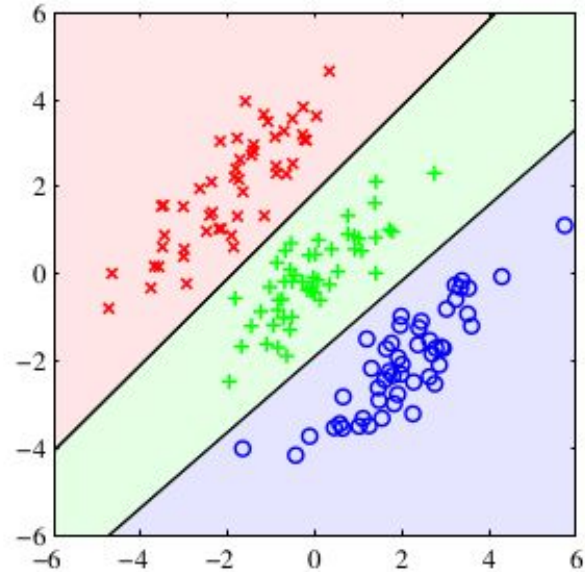
¿cuál clasificador es mejor? **Bishop, Pattern Recognition*

Tipos de clasificación

Clasificación Binaria



Clasificación Multiclase



En clasificación binaria aprenderemos una sola función (clasificador) mientras que en multiclase será k o $k-1$ funciones según el caso.

Ejercicio clasificación

$$\mathbf{x}_{train} = \begin{matrix} & x_1 & x_2 \\ \begin{bmatrix} 1 & 1 \\ 3 & 0.5 \\ 2 & 3 \\ 2.5 & 1.5 \\ 1.5 & 2 \\ 2.5 & 4 \\ 3.5 & 1.5 \\ 4 & 3 \end{bmatrix} & \mathbf{y}_{train} = \begin{bmatrix} -1 \\ -1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \end{matrix}$$

$$f(x) = w^T x + b$$

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \quad \mathbf{b}$$

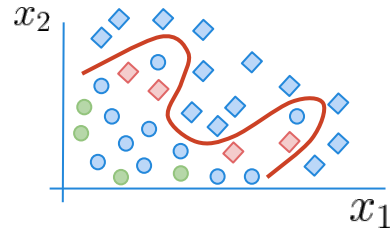
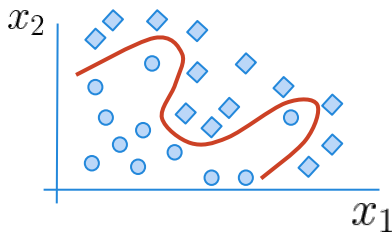
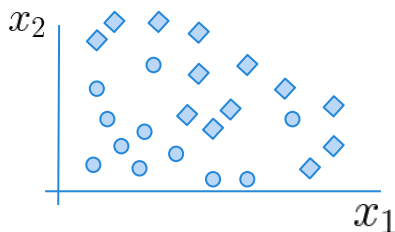
$$n = 8 \quad d = 2 \quad \mathcal{X} \in \mathbb{R}^2$$

Dado el dataset (x,y) de entrenamiento, calcular cuáles deberían ser los valores de los parámetros $\mathbf{w}$ y $\mathbf{b}$ para obtener una función lineal de decisión que clasifique los datos con el menor error. Graficar la solución.

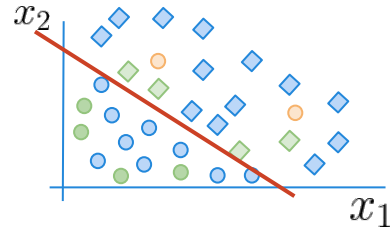
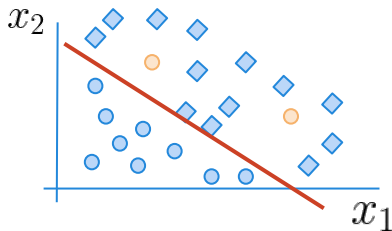
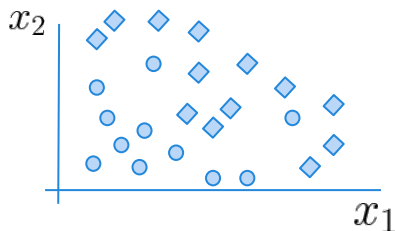
Aprendizaje supervisado: clasificación



Modelo no-lineal



Modelo lineal



Datos de entrenamiento con etiquetas de dos clases.

Funcion de clasificacion: entrenamos un modelo no-lineal y lineal.

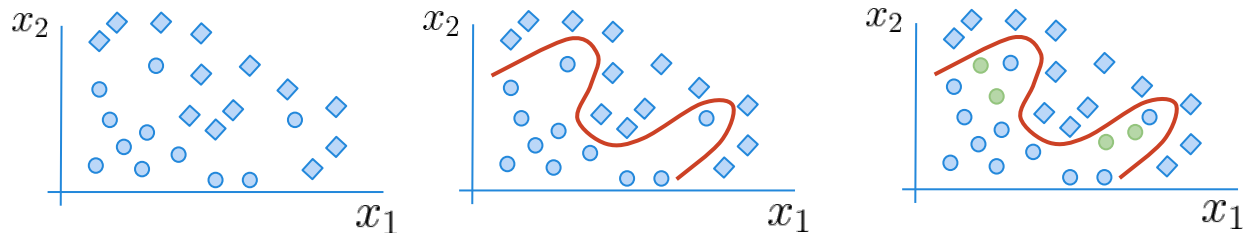
Agregamos 4 muestras nuevas de cada clase y vemos cómo clasifican con el modelo entrenado.

Durante el entrenamiento, el modelo estará expuesto a muchas muestras de “entrenamiento” y sus respectivas etiquetas de manera tal de que aprenda la regla $f'(x)$ que **minimiza el error de clasificación**. Simultáneamente esperamos que luego de entrenar, la regla/función de decisión aprendida en los datos de entrenamiento funcione “bien” para muestras que nunca vió (test). Es decir, que pueda **generalizar** a instancias ‘x’ nunca vistas clasificándolas correctamente.

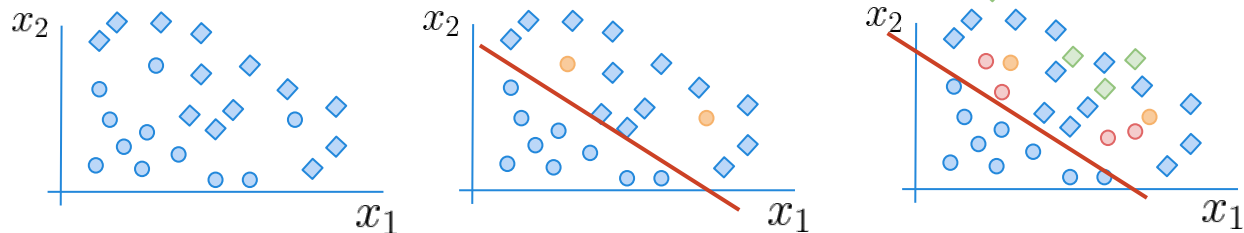
En la figura: cuál modelo tiene menor error en el entrenamiento? cual modelo generaliza mejor? Cual es más complejo? Cual es más sencillo?

Aprendizaje supervisado: clasificación

Modelo no-lineal



Modelo lineal



Datos de entrenamiento con etiquetas de dos clases.

Funcion de clasificación: entrenamos un modelo no-lineal y lineal.

Agregamos 4 muestras nuevas de cada clase y vemos cómo clasifican con el modelo entrenado.

Dados los mismos datos de entrenamiento del caso anterior, volvemos a entrenar los dos modelos: lineal y no lineal. Esta vez con otras muestras nuevas (test). Los resultados de clasificación son distintos en este caso. Cual de los dos modelos es mejor? Cual es la principal diferencia entre ambos modelos? Cual de los dos debemos mantener para el análisis?

Aprendizaje supervisado: clasificación

Clase 1 train



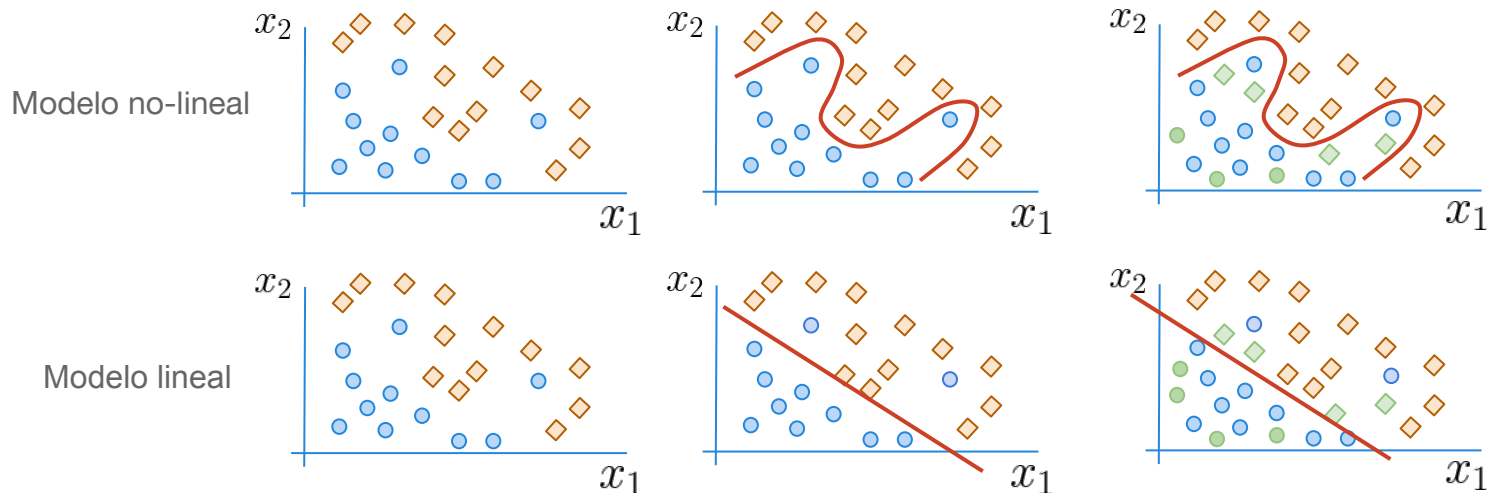
Clase 1 test



Clase 2 train



Clase 2 test



Datos de entrenamiento
con etiquetas de dos clases.

Funcion de clasificacion: con los
datos de entrenamiento
ajustamos un modelo
no-lineal y lineal.

Evaluacion: exponemos el
modelo ya entrenado
con 4 muestras nuevas de
validacion/test de cada clase y
vemos cómo clasifican con el
modelo ya entrenado.

Aprendizaje supervisado: clasificación

Clase 1 train



Clase 1 test



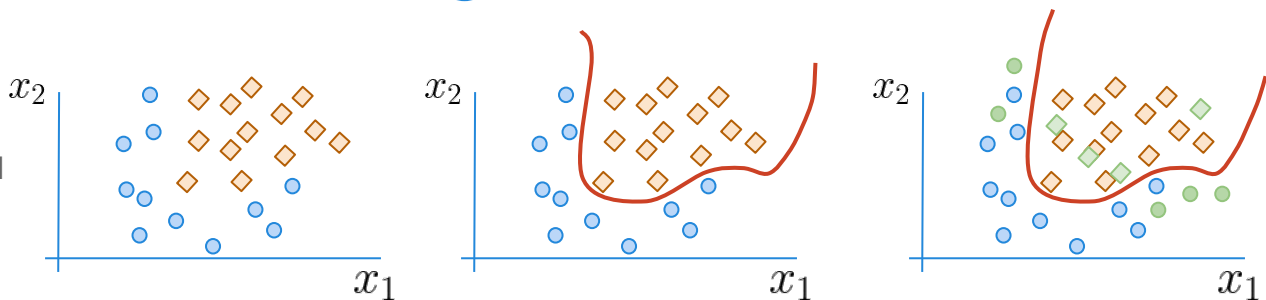
Clase 2 train



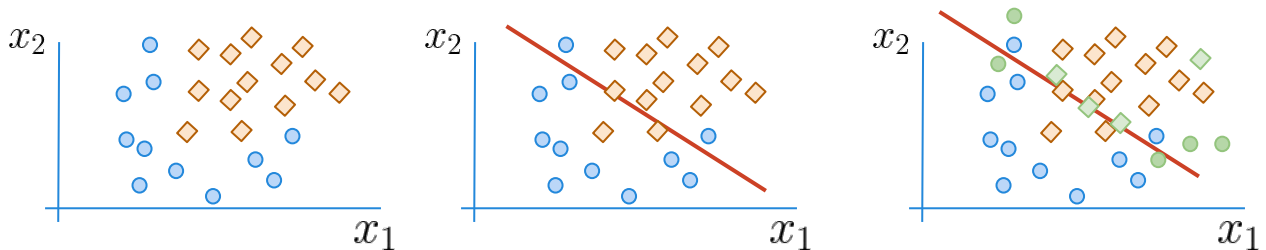
Clase 2 test



Modelo no-lineal



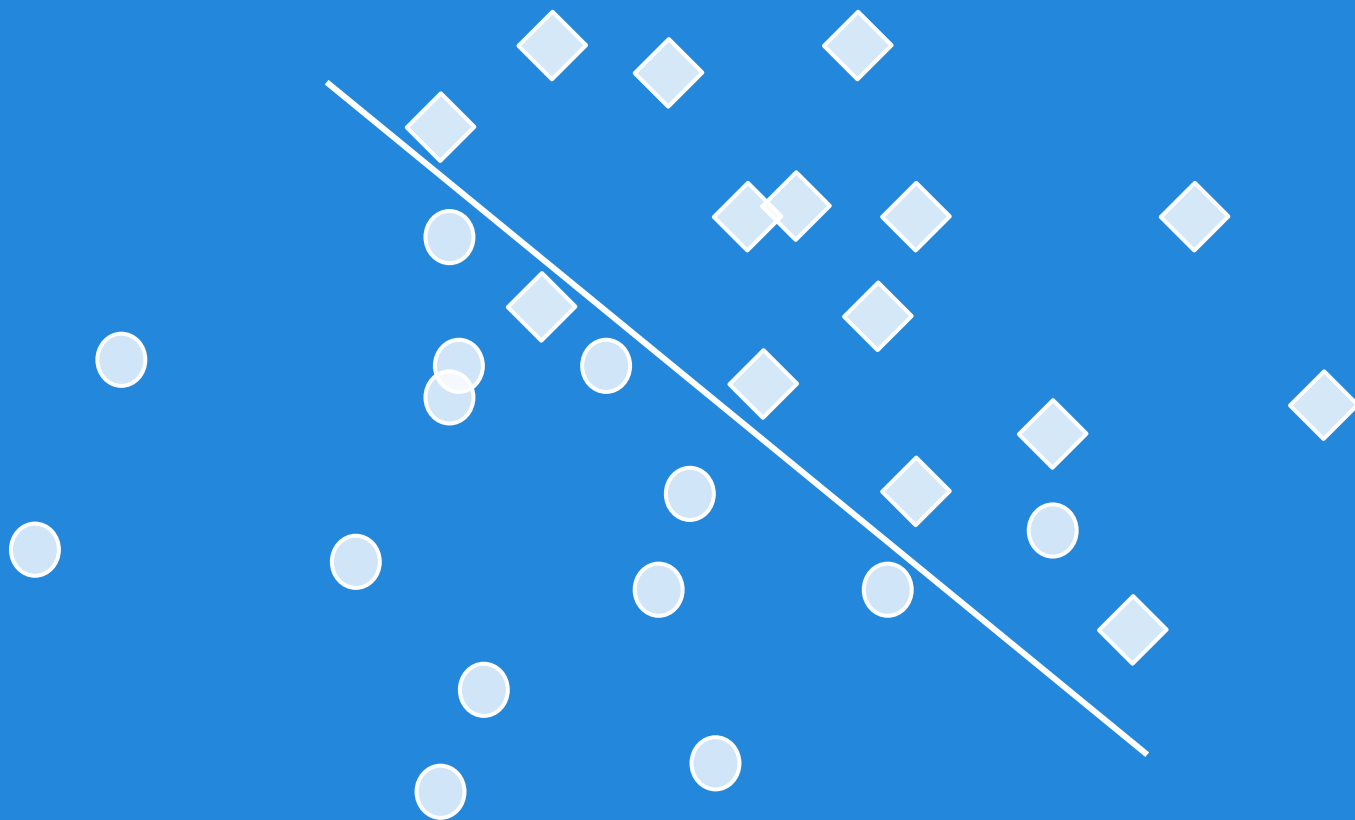
Modelo lineal



Datos de entrenamiento
con etiquetas de dos clases.

Funcion de clasificacion: con los
datos de entrenamiento
ajustamos un modelo
no-lineal y lineal.

Evaluacion: exponemos el
modelo ya entrenado
con 4 muestras nuevas de
validacion/test de cada clase y
vemos cómo clasifican con el



Aprendizaje supervisado: mediciones de desempeño/performance en clasificación

classification results: confusion matrix

Predicted Label			
		Class1 (-)	Class2 (+)
True Label	Class1 (-)	True Negative	False Positive
	Class2 (+)	False Negative	True Positive

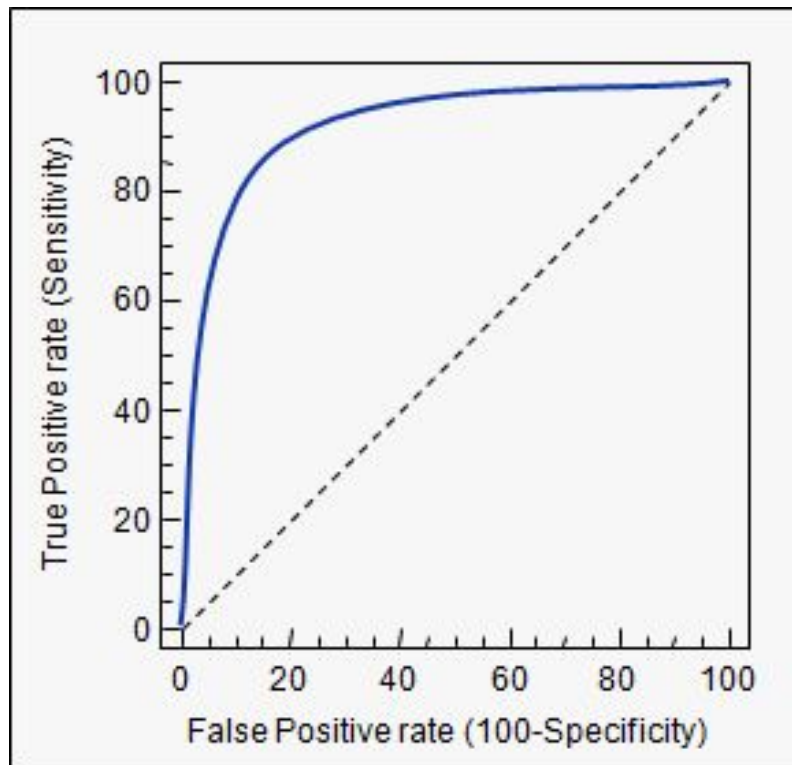
Accuracy = (TN+TP) / Total

Sensitivity (recall) = TP/(TP+FN)

Specificity = TN/(TN+FP)

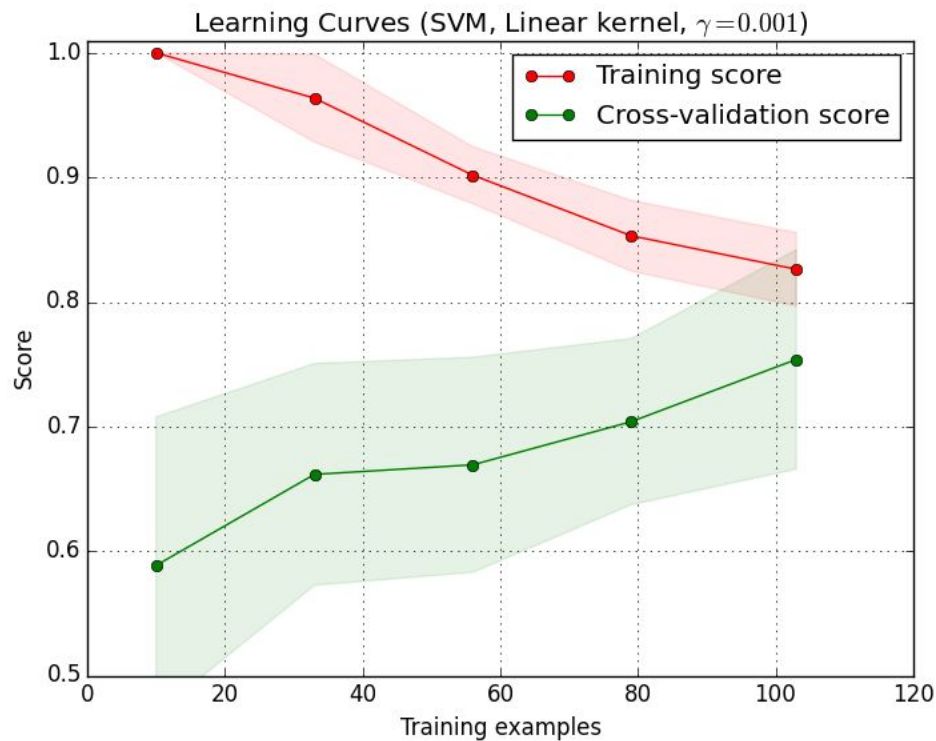
La matriz de confusión es un elemento para evaluar los resultados de la clasificación. En cada posición se cuentan los TP, TN, FP, FN. Luego se obtienen los coeficientes de accuracy, Sensitivity, specificity.

classification results: AUC ROC



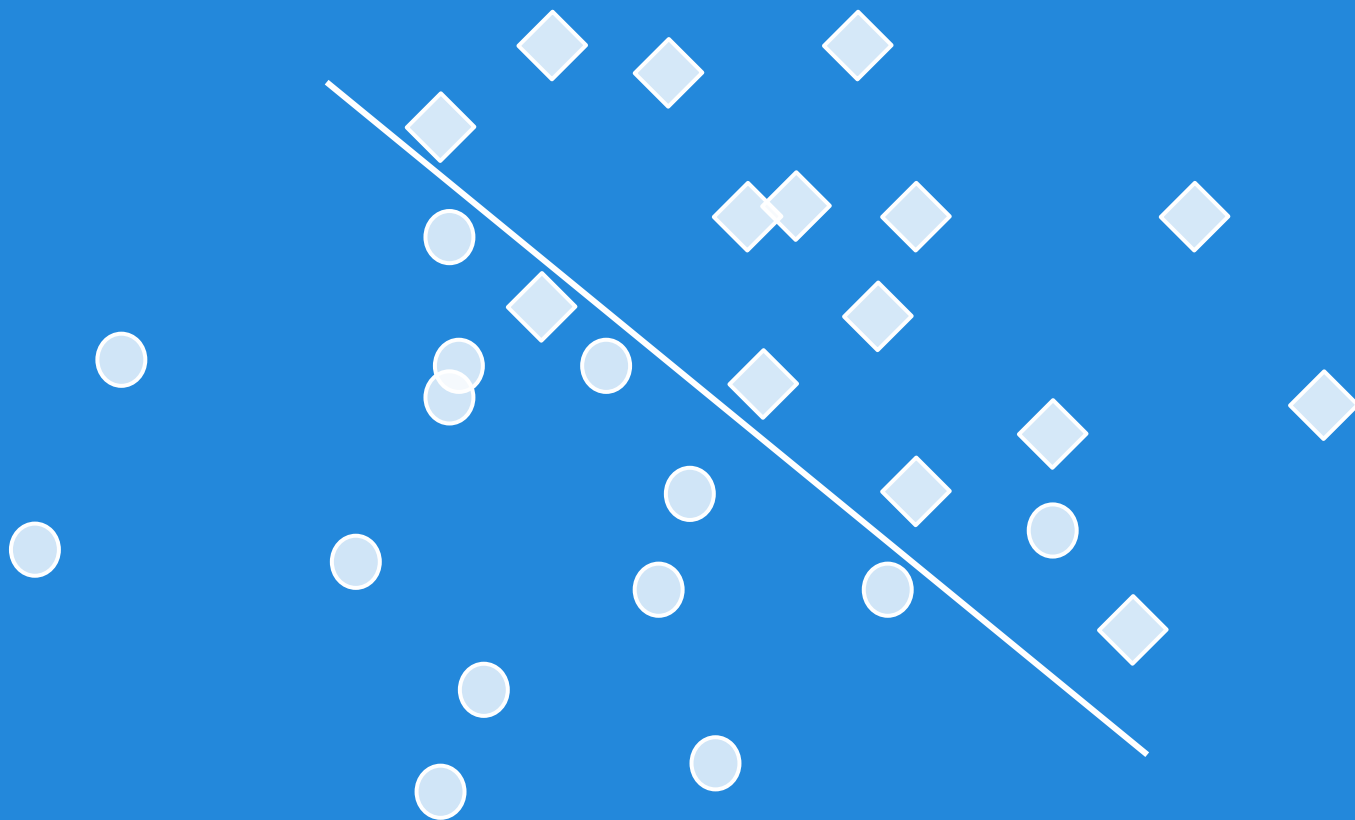
- El área bajo la curva ROC (AUC) da una idea de cuan bueno es mi clasificador independientemente del accuracy.
- Tomando una sola clase, se analizan distintos umbrales de clasificación y se contempla la relación entre TP y FP.
- Cuanto más cerca de 0.5 sea el AUC, más similar a “arrojar una moneda” será mi clasificador. Cuanto más cerca de 1 sea el área bajo la curva, mejor mi clasificador.

Learning Curve



Podemos evaluar el funcionamiento del clasificador con distintas cantidades de muestras de entrenamiento, de menos a más.

La curva de aprendizaje es aquella que se forma al computar el accuracy en train y test en función de la cantidad de muestras de train. Idealmente debería converger a un valor determinado.



Aprendizaje supervisado y Fuente del error: Variance vs Bias

Variance vs Bias

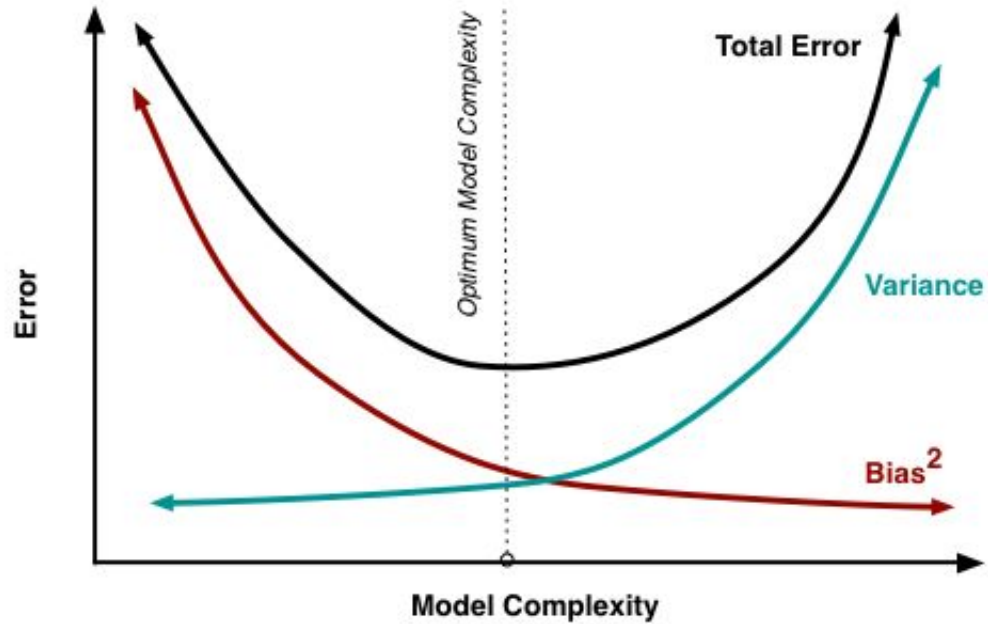
$$\mathcal{L}(y, \hat{y}) = \text{Var}(\hat{f}(x)) + [\text{Bias}(\hat{f}(x))]^2$$

Cuando la etiqueta real y es distinta a la etiqueta asignada por el clasificador $\hat{y}$ la función de pérdida-costo (Loss) $\mathcal{L}$ se incrementará. La causa de que un modelo supervisado tenga error en muestras nuevas se puede causar por su sesgo o por su varianza.

Un modelo tiene alta varianza cuando su predicción varía mucho si lo exponemos a muestras distintas provenientes de la misma población. En general la varianza se observa en modelos complejos (polinomios de alto grado), y está asociada al ‘sobre-ajuste’ (overfitting).

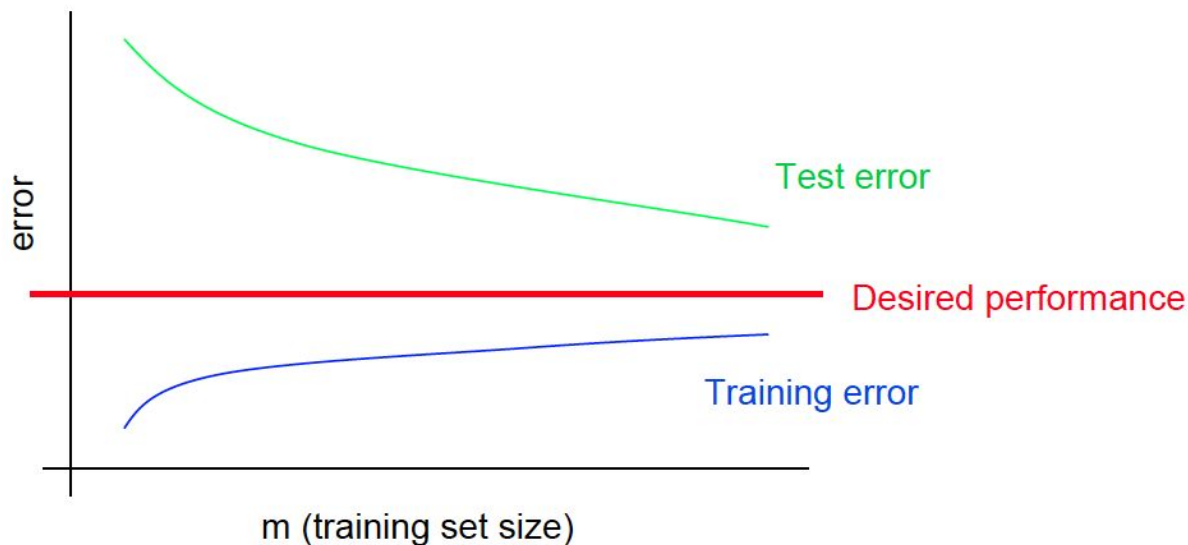
Los modelos con alto sesgo son aquellos que frente a distintos datos provenientes de la misma población no modifican mucho su predicción aunque en general ese valor de predicción no suele ser aceptable. El sesgo está asociado a ‘sub-ajuste’ (underfitting) y se suele observar en modelos muy sencillos (por ejemplo modelos lineales).

Variance vs Bias



*Andrew Ng.

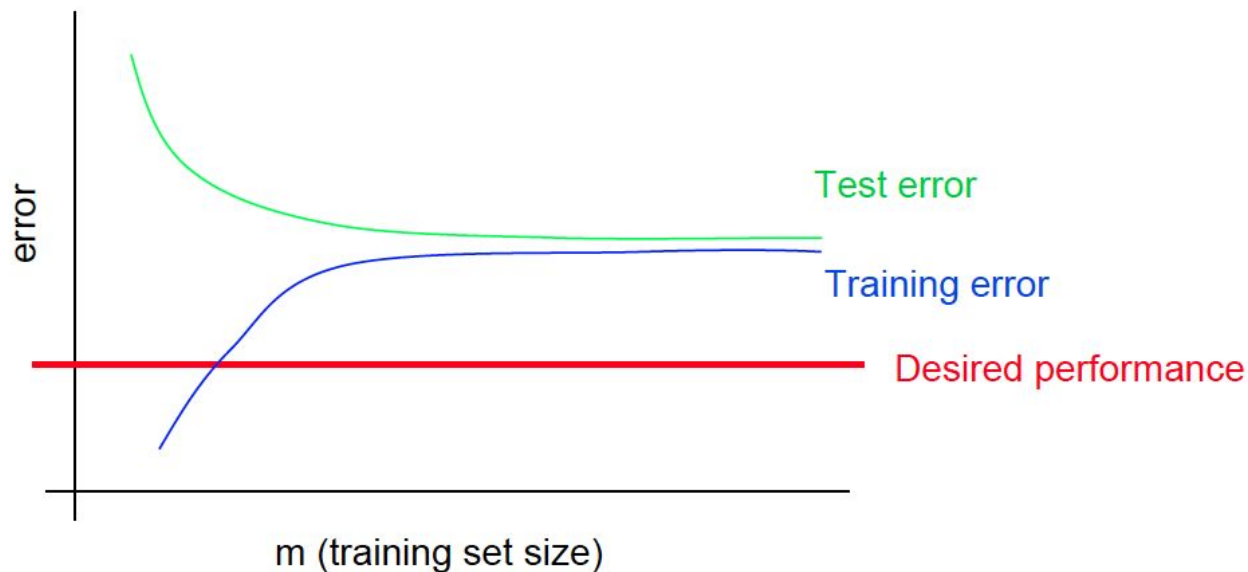
Learning curve: high variance



Alta varianza puede estar relacionada a alta complejidad del modelo.

*Andrew Ng.

Learning curve: high bias



Alto sesgo puede estar relacionado a una falta de complejidad del modelo.

*Andrew Ng.

Momento del meme

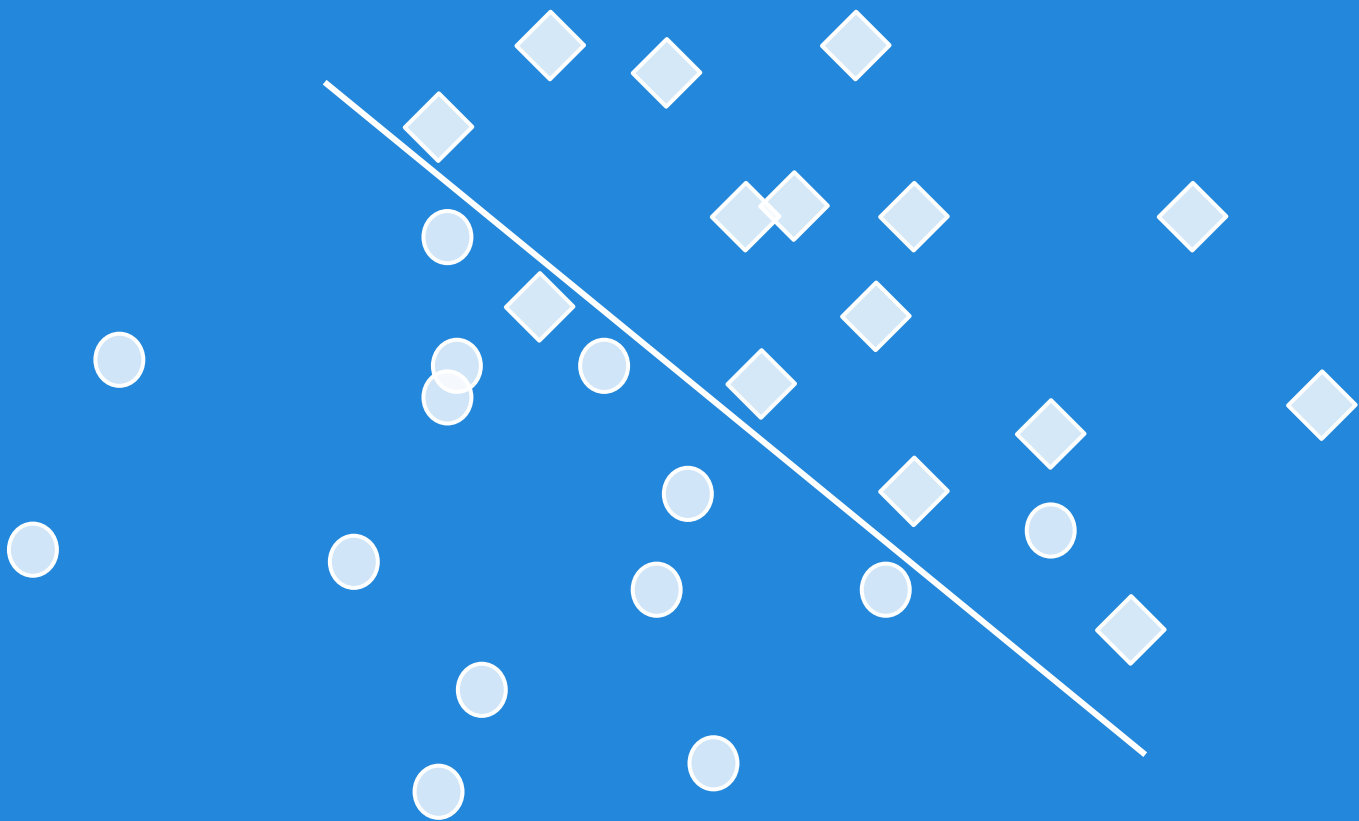
Odio la Estadística 



@Oodioestadistica



@Odiolaestadistica



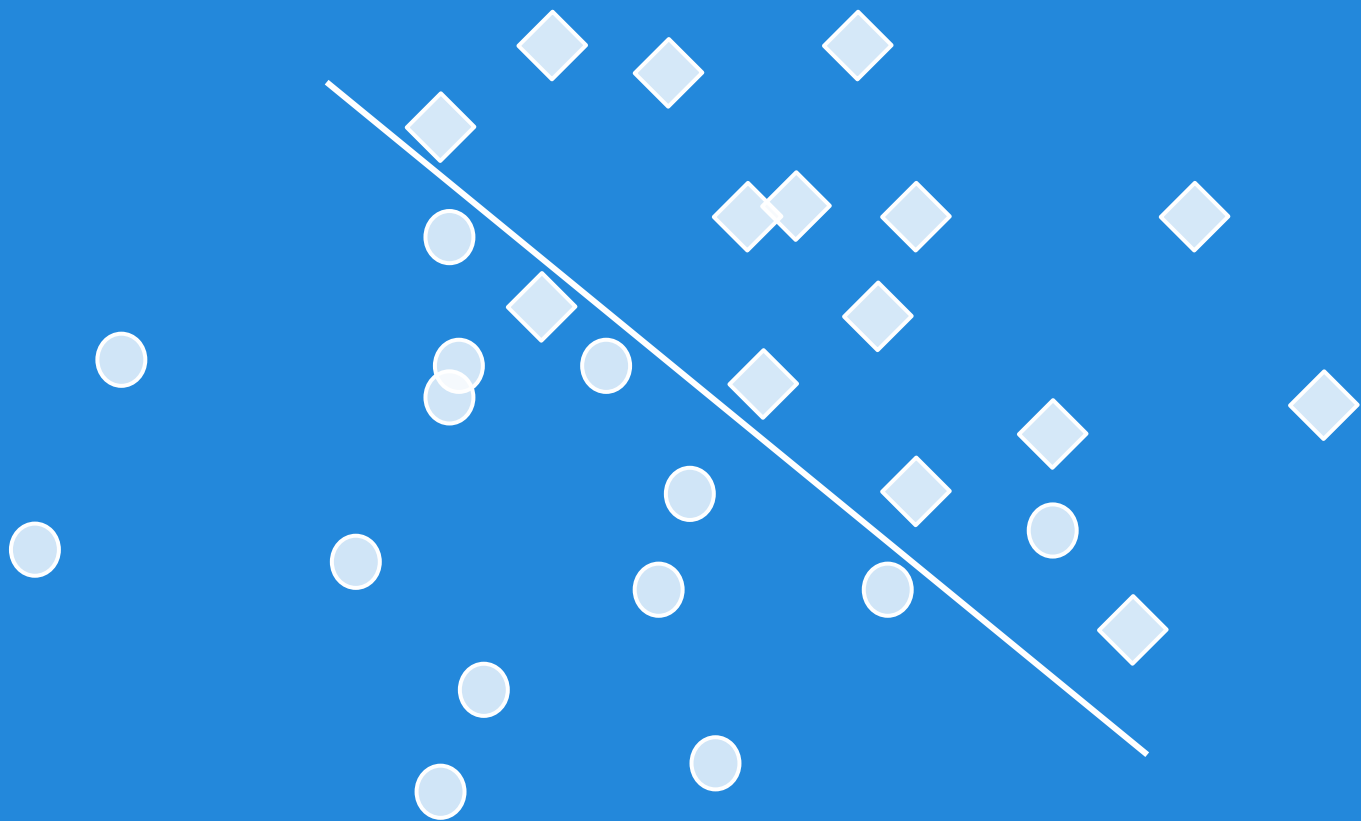
Aprendizaje supervisado: modelos de clasificación

Clasificadores

Vamos a estudiar dos modelos de clasificación

- KNN (esta clase)
- Logistic Regression

Cada uno tendrá ventajas y debilidades respecto a los otros. Hay decenas de clasificadores que por razones de tiempo no incluimos en este curso.



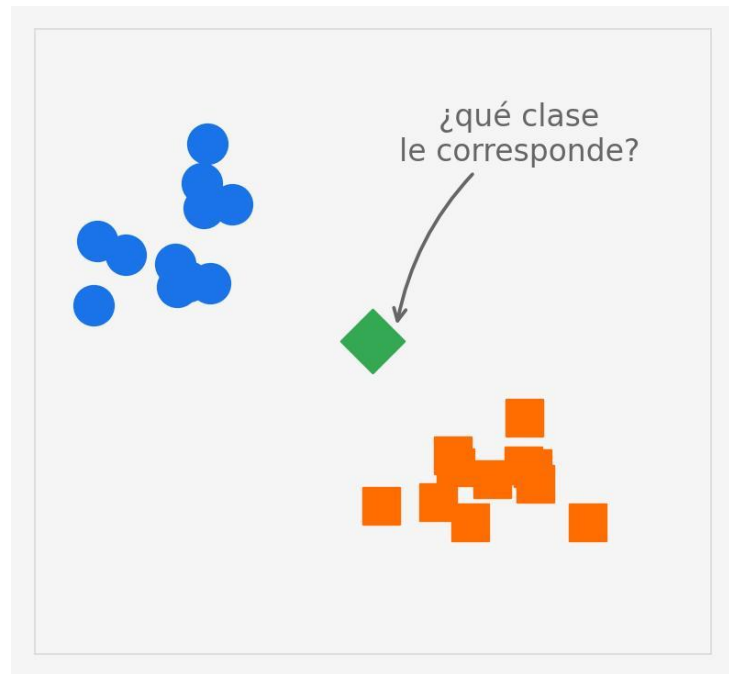
Aprendizaje supervisado: KNN

KNN – La idea intuitiva

Tenemos un conjunto de datos ya etiquetados y llega una muestra nueva, sin etiqueta. ¿Qué clase le asignamos?

- La intuición: cosas parecidas suelen pertenecer a la misma clase.
- Buscamos las muestras conocidas más parecidas a la nueva.
- Le asignamos la clase que aparece más veces entre esos vecinos.

Eso es KNN. El resto de la clase es precisar “parecido” y “cuántos vecinos”.



KNN – K vecinos más cercanos

KNN (K-Nearest Neighbors) es uno de los clasificadores más simples e intuitivos del aprendizaje supervisado.

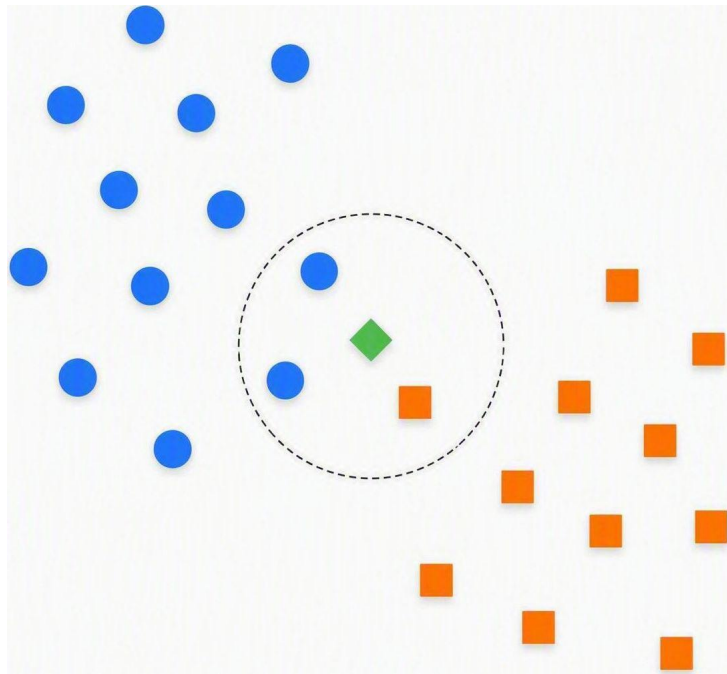
- Basado en instancias: no construye una fórmula, sino que “memoriza” todo el conjunto de entrenamiento.
- “Lazy learning” (aprendizaje perezoso): casi no hay fase de entrenamiento; el trabajo se hace al momento de predecir.
- No paramétrico: no asume ninguna forma para la frontera de decisión.
- Para clasificar una muestra nueva, mira las K más parecidas y decide por mayoría de votos.

Sirve tanto para clasificación como para regresión.

KNN – ¿Cómo funciona?

Para predecir la clase de una muestra nueva, el algoritmo sigue cuatro pasos:

- 1. Calcular la distancia a todas las muestras de entrenamiento.
- 2. Ordenar y seleccionar las K más cercanas: sus “vecinos”.
- 3. Ver a qué clase pertenece cada uno de esos K vecinos.
- 4. Asignar la clase mayoritaria (voto por mayoría).



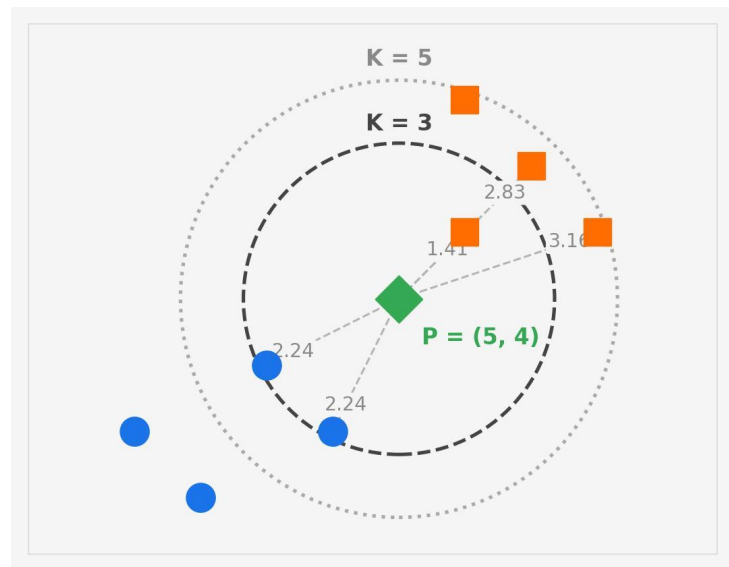
KNN – Ejemplo numérico paso a paso

$P = (5, 4)$. Distancias euclídeas a las 8 muestras de entrenamiento:

- naranja (6,5) $\rightarrow 1.41$
- azul (3,3) $\rightarrow 2.24$ y azul (4,2) $\rightarrow 2.24$
- naranja (7,6) $\rightarrow 2.83$
- naranja (8,5) $\rightarrow 3.16$ y naranja (6,7) $\rightarrow 3.16$
- azul (2,1) $\rightarrow 4.24$ y azul (1,2) $\rightarrow 4.47$

Con $K = 3$: 2 azules vs 1 naranja $\rightarrow$ azul

Con $K = 5$: 2 azules vs 3 naranjas $\rightarrow$ naranja



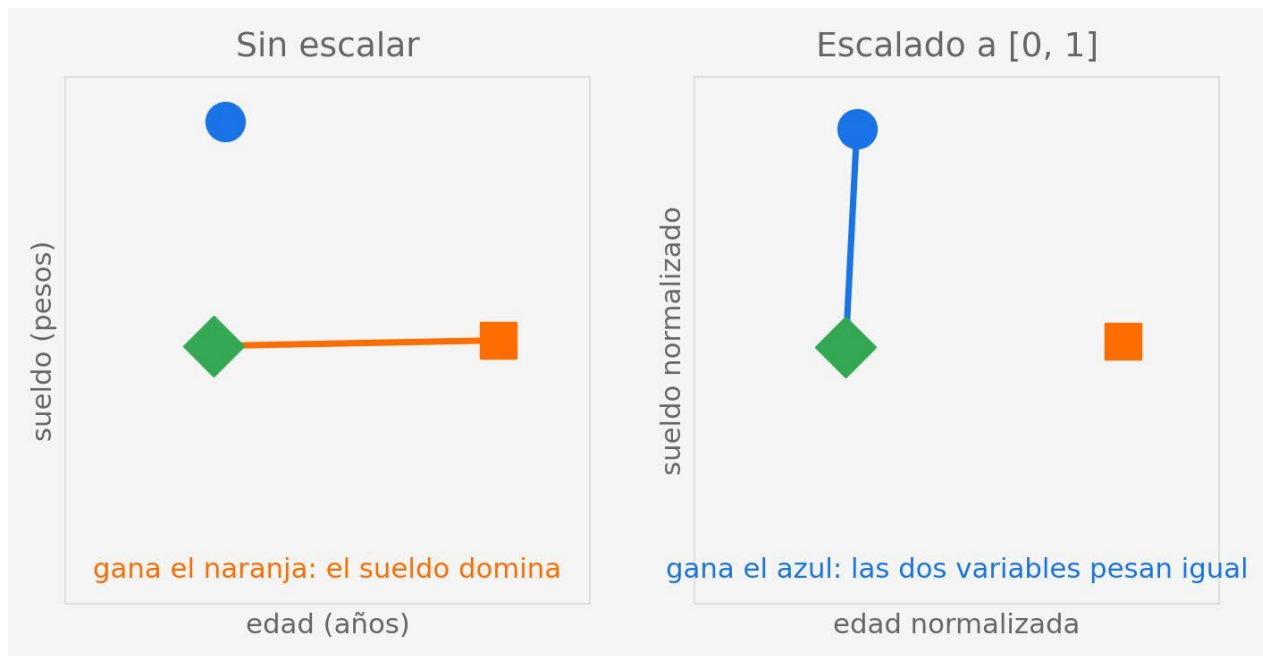
KNN – Hay que escalar las variables

KNN suma diferencias de todas las variables, así que la de mayor magnitud domina la distancia y las demás quedan ignoradas.

- Min-Max lleva cada variable al rango $[0, 1]$.
- Z-score la deja con media 0 y desvío 1.
- El escalador se ajusta SÓLO con los datos de entrenamiento y después se aplica a test. Ajustarlo con todo el dataset es fuga de datos.

KNN – El efecto de escalar

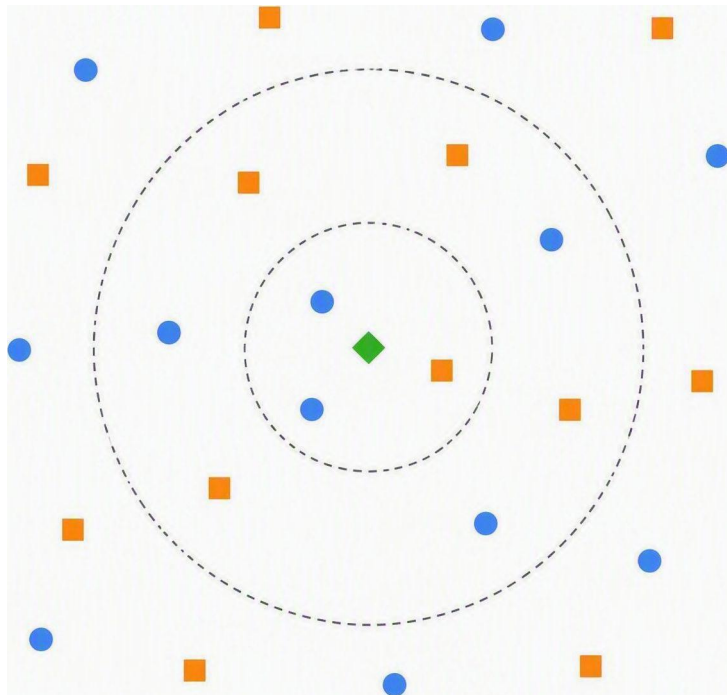
Edad en años (20–60) y sueldo en pesos (500k–900k). Escalar cambia quién es el vecino más cercano, y por lo tanto la predicción:



KNN – La elección de K

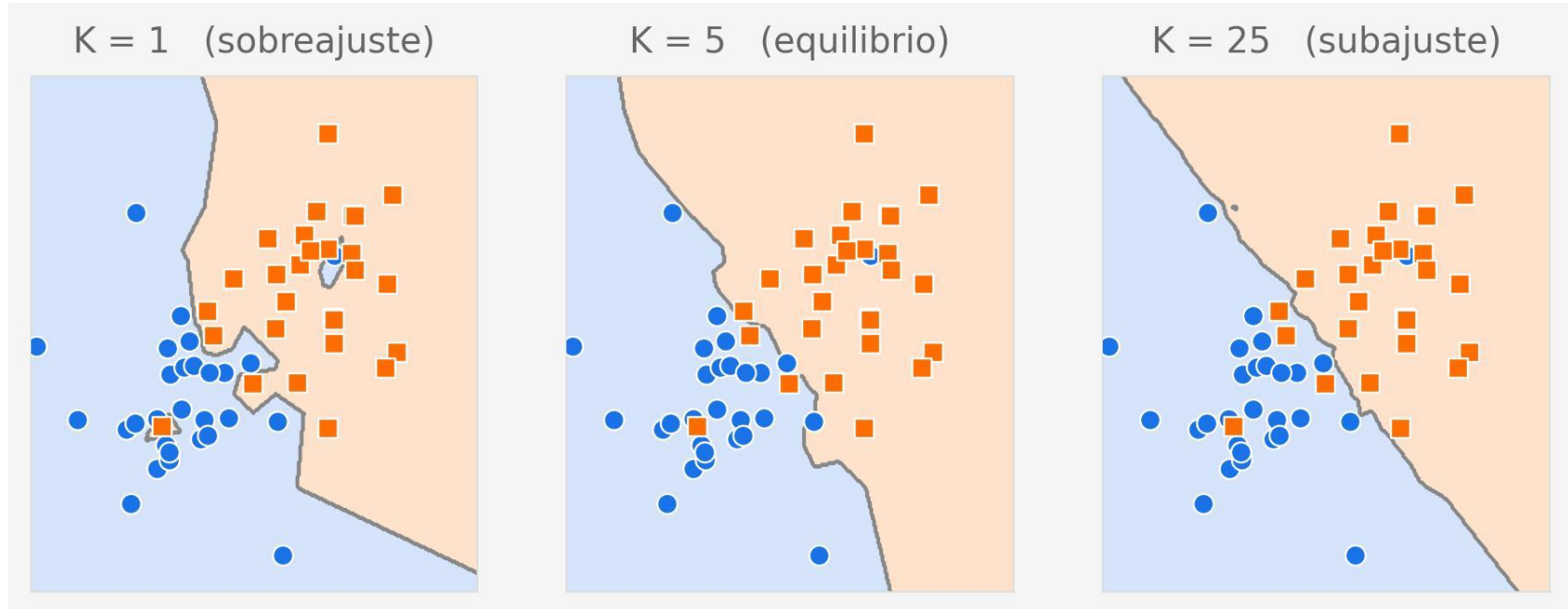
K es el hiper-parámetro clave del modelo:

- K muy chico ($K=1$): sigue el ruido de cada punto, sobreajusta.
- K muy grande: suaviza de más y pierde la estructura local, subajusta.
- Se elige con validación cruzada, no a ojo.
- Conviene K impar en problemas binarios para evitar empates.



KNN – Qué le hace K a la frontera

La misma nube de puntos con tres valores de K. El fondo indica qué clase predice el modelo en cada región:



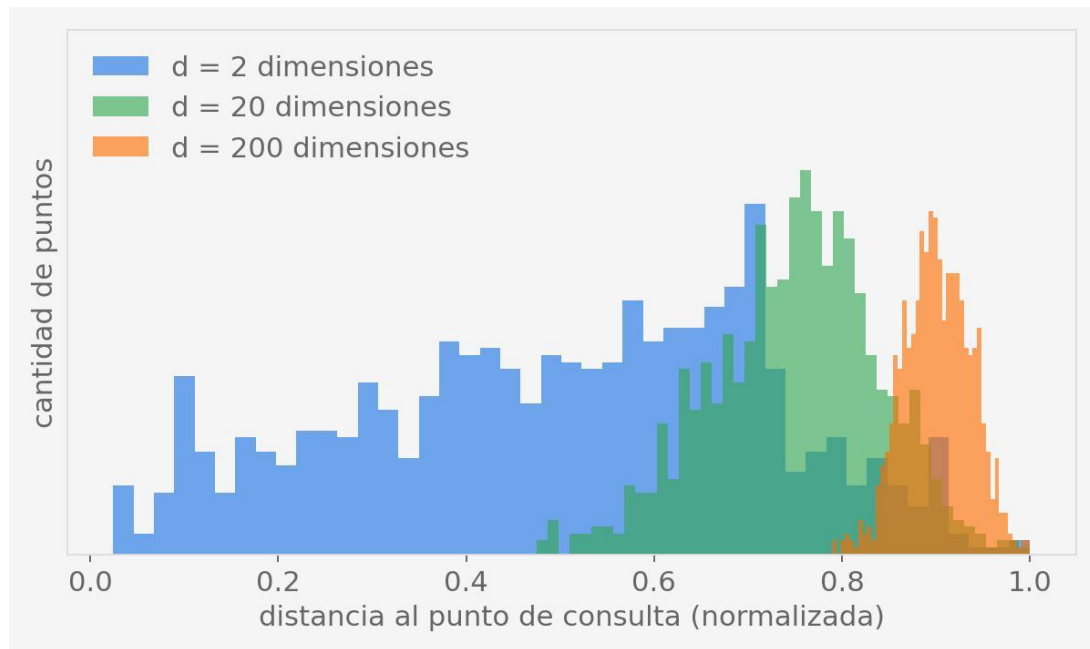
KNN – Maldición de la dimensionalidad

Al crecer la cantidad de variables, todas las distancias se parecen entre sí y “vecino más cercano” deja de significar algo.

- Con muchas dimensiones el espacio queda casi vacío: harían falta exponencialmente más datos para cubrirlo.
- Las variables irrelevantes suman ruido a la distancia y degradan el modelo.
- Qué hacer: seleccionar variables, reducir dimensión con PCA.

KNN – Distancias en alta dimensión

Distancia de 900 puntos aleatorios a un punto de consulta, en 2, 20 y 200 dimensiones:



KNN: Hiper Parametros

Los hiper-parámetros más importantes de KNN son

- **K = “Cantidad de vecinos”** , el más importante. Define cuántos vecinos se consideran para realizar la predicción.
- **Metric**: Define cómo se calcula la distancia o similitud entre las observaciones. Distancia Euclidean, Manhattan, otra.
- **Weights**: Define cuánto influye cada vecino en la predicción según su distancia.

KNN – Implementación en scikit-learn

```
# Samples and features
X = np.array([[-1, -1], [-2, -1], [1, 1], [2, 1]])

# Labels (two classes)
y = np.array([1, 1, 2, 2])

# import KNN model
from sklearn.neighbors import KNeighborsClassifier

# define KNN model
knn = KNeighborsClassifier(n_neighbors=3)

# fit KNN on training data
knn.fit(X, y)

# predict a new independent sample
y_prediction = knn.predict([[-0.8, -1]])

# print prediction
print(y_prediction)

[1]
```

KNN – Ventajas y desventajas

Ventajas

- Simple de entender e implementar; no asume ninguna distribución de los datos.
- Captura fronteras de decisión complejas y no lineales.
- Sirve para clasificación y regresión, y es multiclase de forma natural.

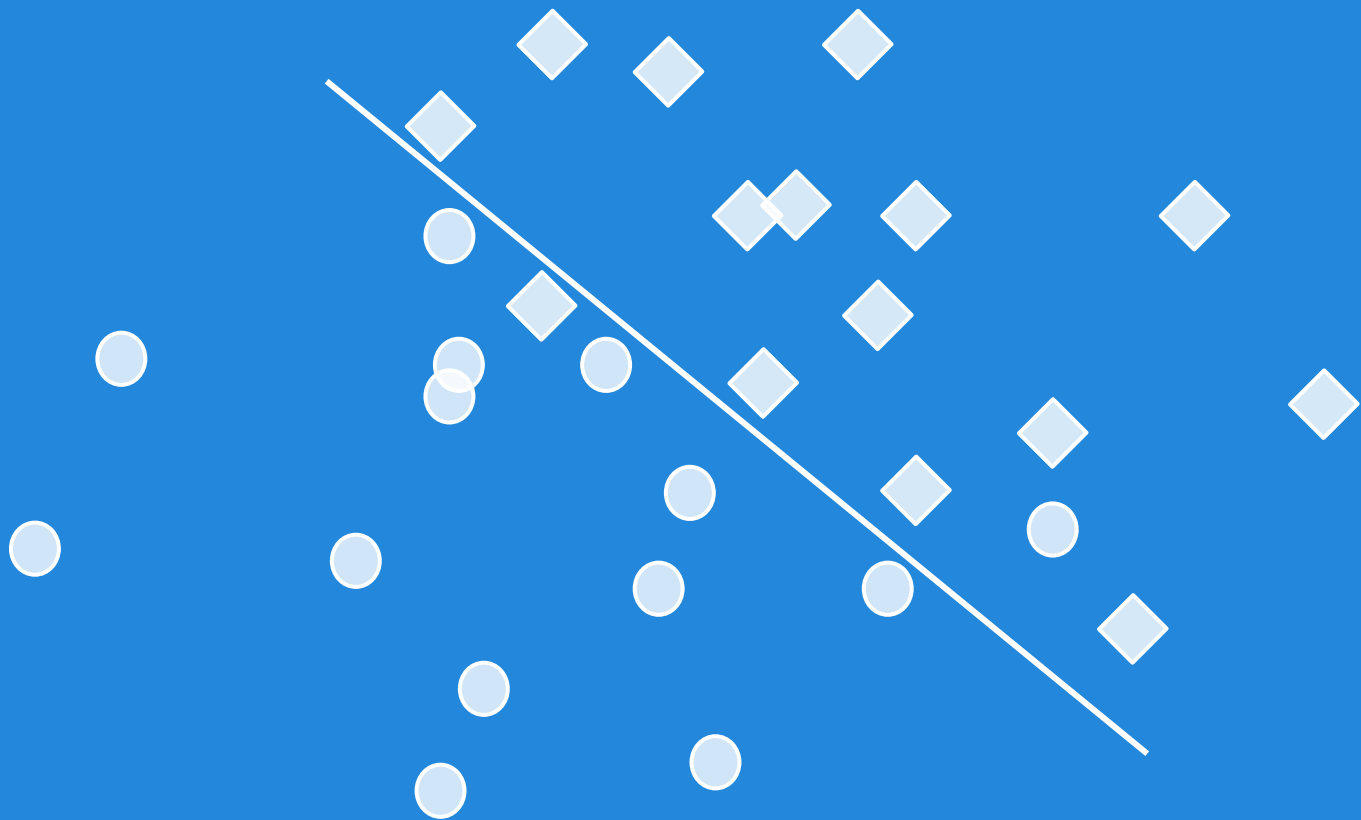
Desventajas

- Predicción costosa: debe compararse con todo el dataset, por lo que es lento con muchos datos.
- Sensible a la escala y a variables irrelevantes; sufre la maldición de la dimensionalidad.
- Se desbalancea si una clase es mucho más frecuente que las otras.

KNN – También sirve para regresión

Misma mecánica, pero en lugar de votar se promedia el valor de los K vecinos más cercanos. Con $K = 1$ la predicción copia el ruido; al aumentar K se suaviza.





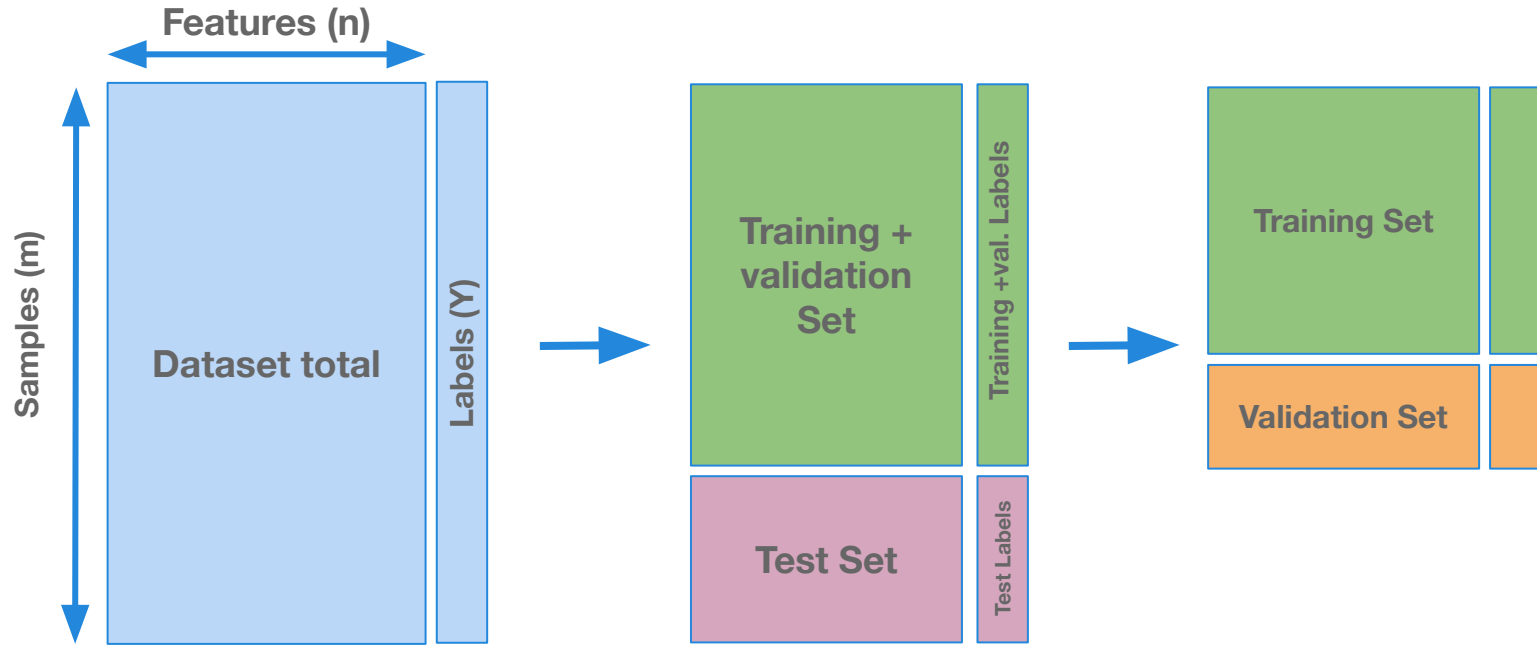
Aprendizaje supervisado: cross validation & hyperparameter tuning

Hiper-parámetros

$f_{w,\lambda}(x)$ funcion de decision
 w parametros
 λ hiper-parametros

Los modelos están caracterizados por parámetros que son aprendidos (encontrados) durante el entrenamiento al ser expuestos a los datos. Adicionalmente los clasificadores tienen hiper-parámetros que definen la familia de funciones que se pueden aprender. Por ejemplo, un hiper-parámetro podría ser el grado de una función polinomial. Los hiper-parámetros no son aprendidos por un algoritmo, son prefijados por el usuario. Los hiperparametros son útiles para poder determinar la complejidad y flexibilidad del clasificador. Por medio de una técnica llamada validación cruzada (cross validation) + búsqueda de grilla (grid search) determinaremos cual la configuración del hiper parámetro que minimiza el error de clasificación.

Train, Validation, Test sets.



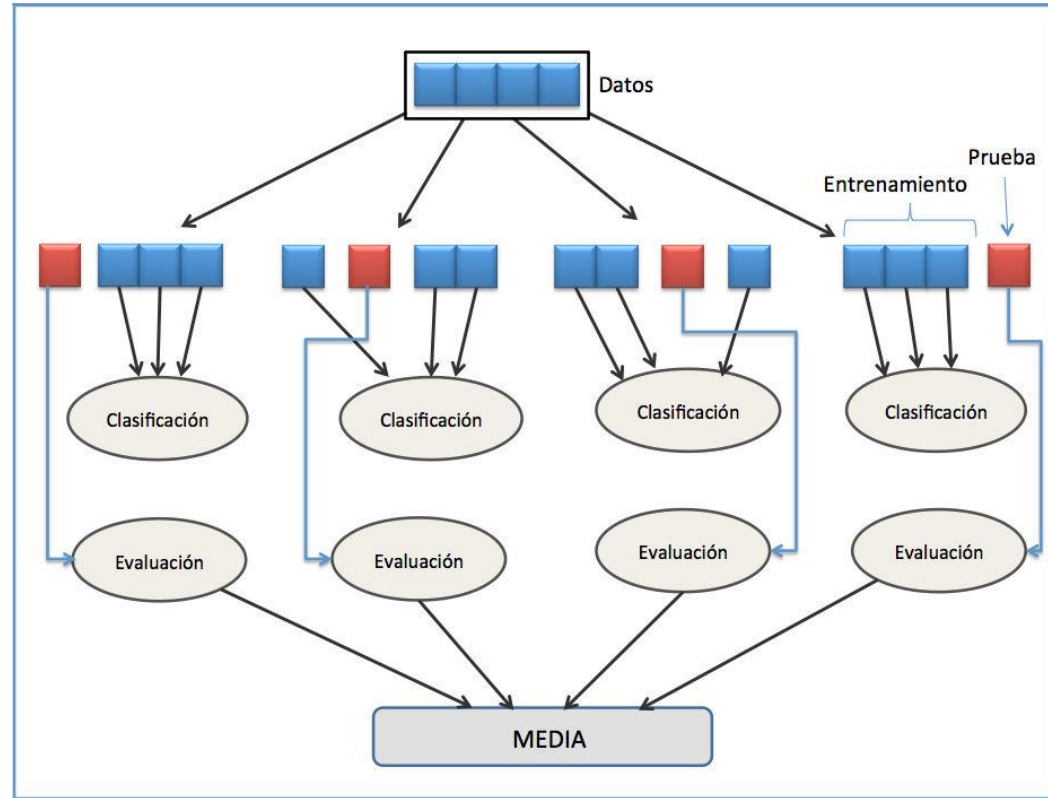
El clasificador aprenderá la regla de decisión utilizando el train set (samples + labels). Luego clasificará las muestras de test (sin mirar las labels de test) y se medirá la exactitud de clasificación en testeo.

Cross - Validation en training set

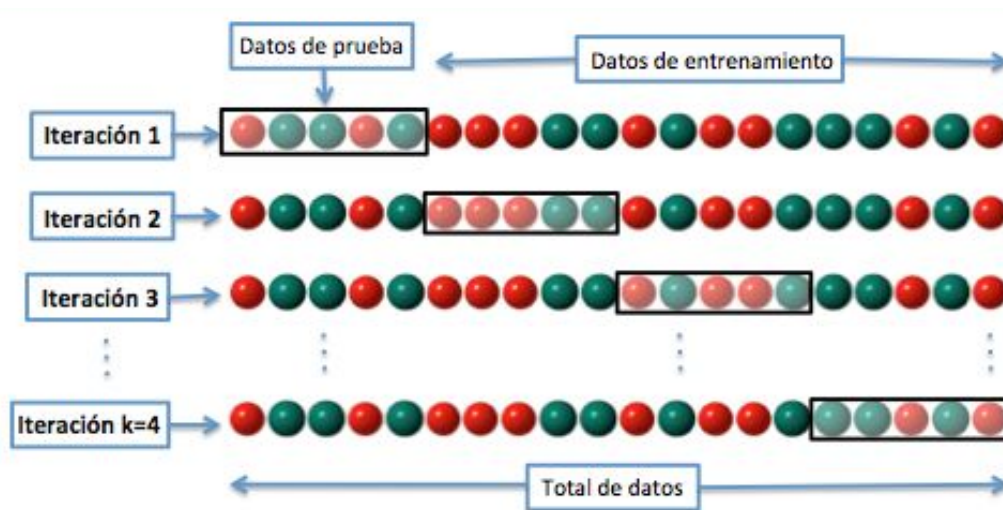
“**Cross validation**” (CV) se realiza con las muestras de entrenamiento. Consiste en dividir nuestro training set en **K folds** (K porciones) e iterar K veces.

En cada iteración, una porción se utiliza como validación independiente y el resto como train. En cada iteración se entrena un modelo con train y se evaluará el resultado de clasificación con validación. Luego se realizará un promedio de la exactitud de clasificación de las k iteraciones.

Cross validation sirve para poder estimar el error estadísticamente. Además si existen varios hiperparametros a cada uno se estima su error por cross validation y se preserva el hiper parámetro que menor error promedio de cross validation genere.



Cross Validation



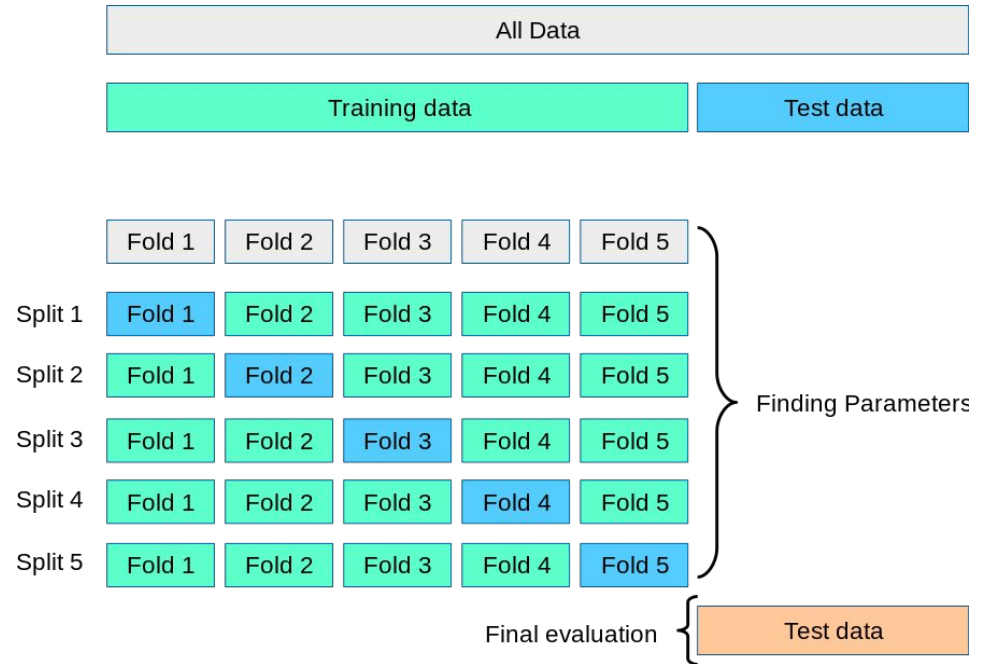
Veremos que nuestros modelos clasificadores a entrenar tendrán distintos “hiper-parámetros a elegir”, por ejemplo si deseamos que nuestro modelo clasificador sea complejo o no, o si queremos que sea “exigente” con cada error de clasificación o permisivo en mayor o menor grado.

Estos hiper-parámetros son elegidos inicialmente por el usuario. Luego por validación cruzada determinaremos el mejor hiper-parámetro para clasificar.

Cross Validation

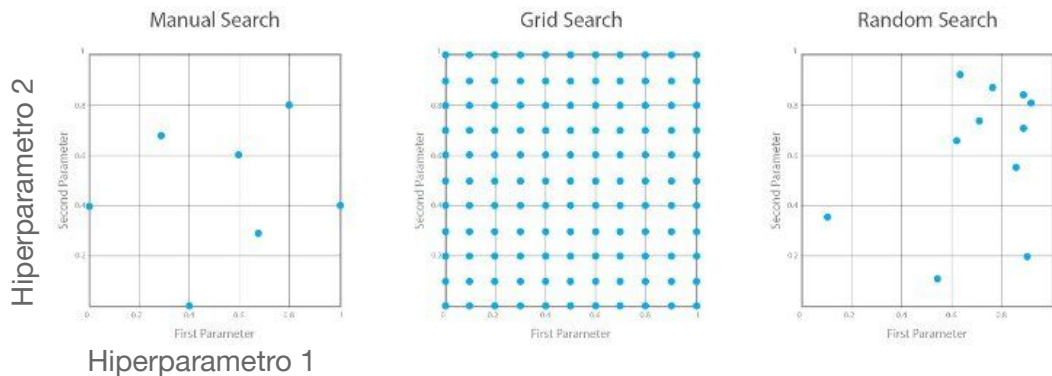
El cross validation también nos da una idea de como funciona el modelo propuesto frente a distintas particiones train-valid de mis datos. En general un buen modelo debería “clasificar aceptablemente bien” en todas las particiones.

Es una manera de “sincerar” si el modelo funciona bien en distintos escenarios y no solo depender de la suerte de nuestra partición.



Grid Search

- Los modelos de clasificación que utilizaremos consistirán de **hiper-parámetros** que el usuario debe seleccionar. Estos determinarán la regla de decisión y por ende la performance del modelo.
- Para saber qué hiper-parámetros seleccionar, lo que haremos es generar una lista de los mismos y probaremos todas las combinaciones posibles de ellos (**grid-search**)
- La combinación de hiper-parámetros que mayor **Train Accuracy** promedio genere durante el cross-validation será la que usaremos para testear el modelo.



Pipeline: Train, Validate, Test Model

**Dividir
Train y
Test**

**Cross Validation &
Grid Search con
Train Set (utilizando
Xtrain e Ytrain)**

**Selección del
mejor modelo e
hiperparámetros**

**Clasificar muestras
de Test (Xtest) sin
mostrarle al modelo
las Ytest.**

**Evaluar
resultados de
clasificación en
test (comparar
Ypred vs Ytest)**